

Network Computing

(058172)



POLITECNICO
MILANO 1863

Congestion Control

Some parts of the slides are inspired from Mohammad Alizadeh's talk on DCTCP and pFabric

Content

- Recap on TCP
- Datacenter TCP (DCTCP)
- pFabric
- Congestion control evolution

Before we start

fun fact

Stanford | ENGINEERING

Search this site



[Admission & Aid](#)

[Students & Academics](#)

[Faculty & Research](#)

[Get Involved](#)

[About](#)

Stanford biologist and computer scientist discover the 'anternet'

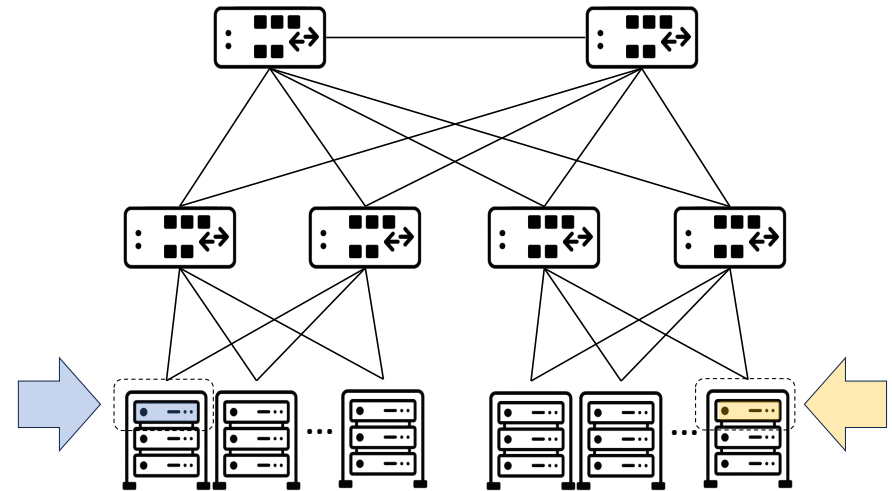
A Stanford ant biologist and a computer scientist has revealed that the behavior of harvester ants as they forage for food mirrors the protocols that control traffic on the Internet.

August 24, 2012 | Bjorn Carey [f](#) [t](#) [in](#) [✉](#) [📄](#)

<https://engineering.stanford.edu/magazine/article/stanford-biologist-and-computer-scientist-discover-anternet>

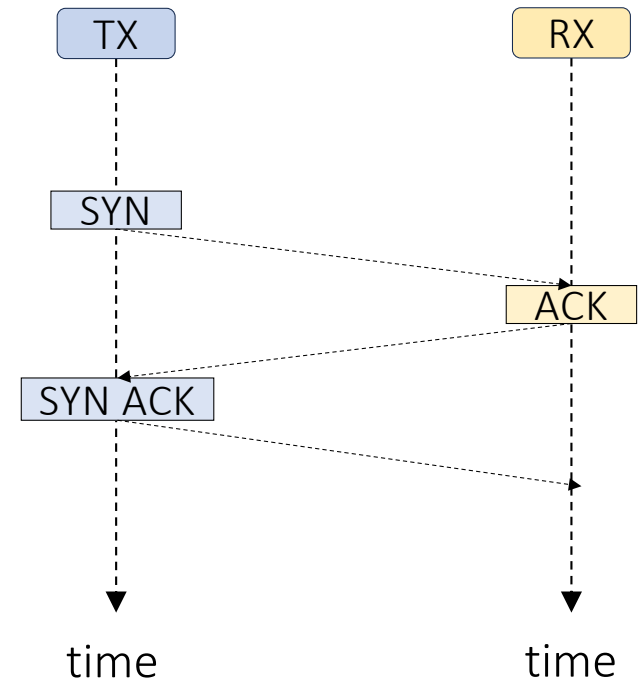
Transmission Control Protocol (TCP) primer

- Consider a scenario where the server marked by the blue arrow must reliably communicate with the server marked by the yellow arrow
- This can be done with TCP that guarantee reliable, in order, byte stream delivery



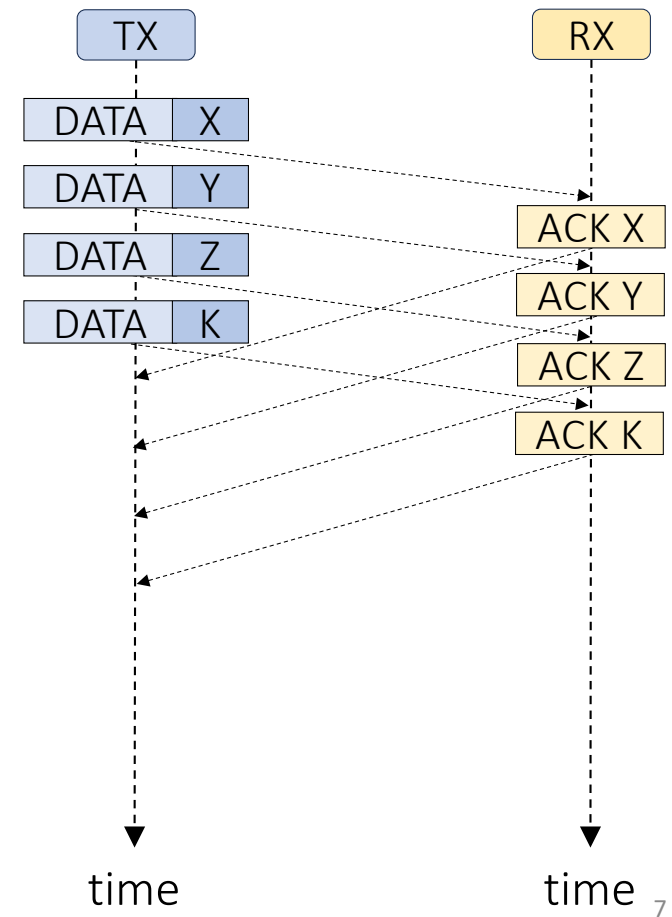
Transmission Control Protocol (TCP) primer

- Before data exchange the connection setup must happen
 - three way handshake: SYN, SYN-ACK, ACK



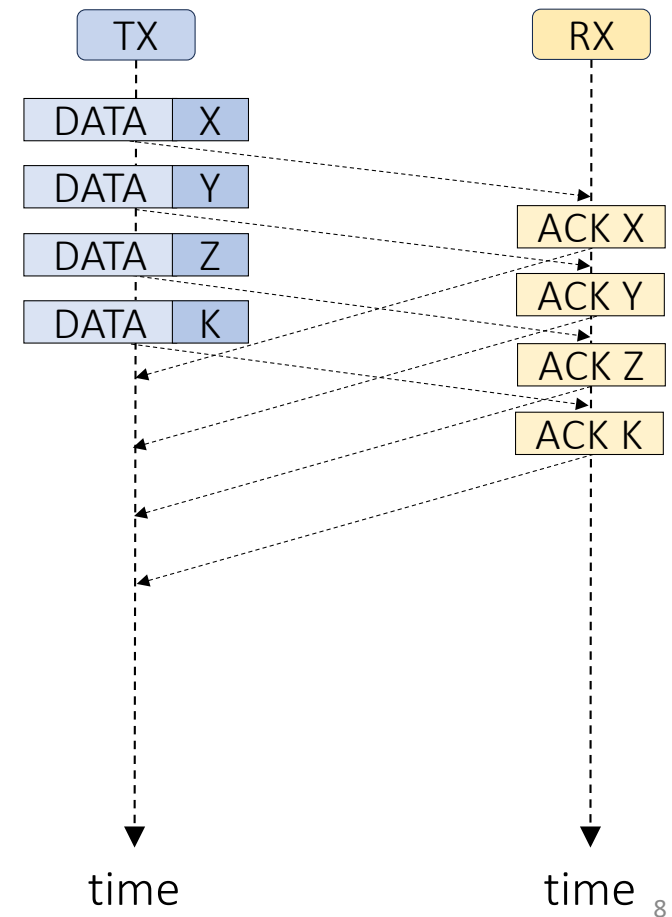
Transmission Control Protocol (TCP) primer

- Before data exchange the connection setup must happen
 - three way handshake: SYN, SYN-ACK, ACK
- The TCP stack in the blue server then splits the data in segments as network works with limited-sized packets
- It then send them over the network



Transmission Control Protocol (TCP) primer

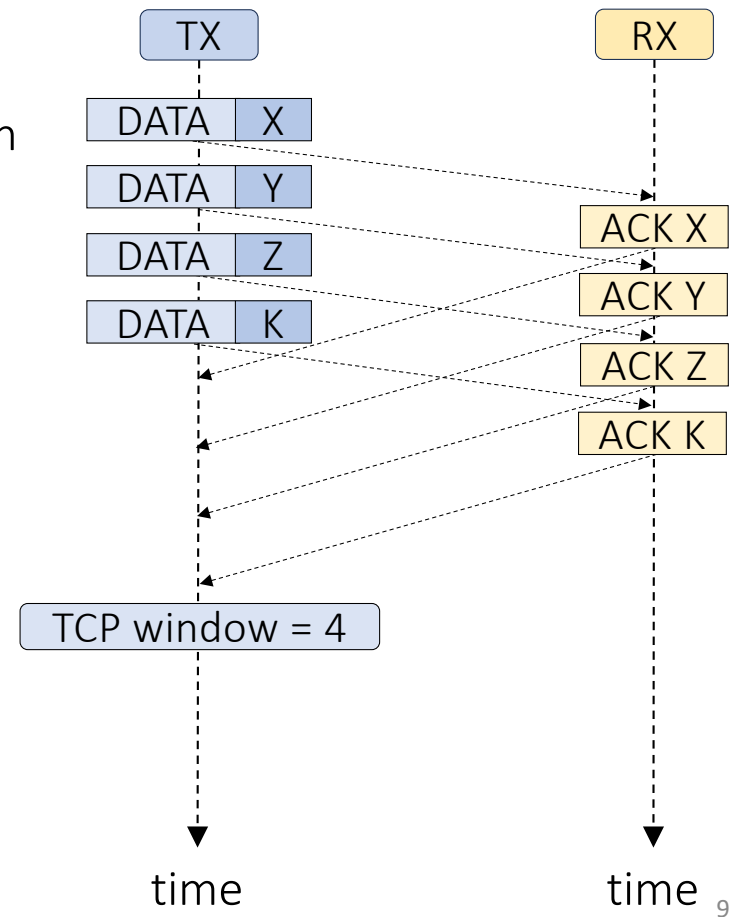
- Before data exchange the connection setup must happen
 - three way handshake: SYN, SYN-ACK, ACK
- The TCP stack in the blue server then splits the data in segments as network works with limited-sized packets
- It then send them over the network
- How many in-flight (without an ACK)?



Transmission Control Protocol (TCP) primer

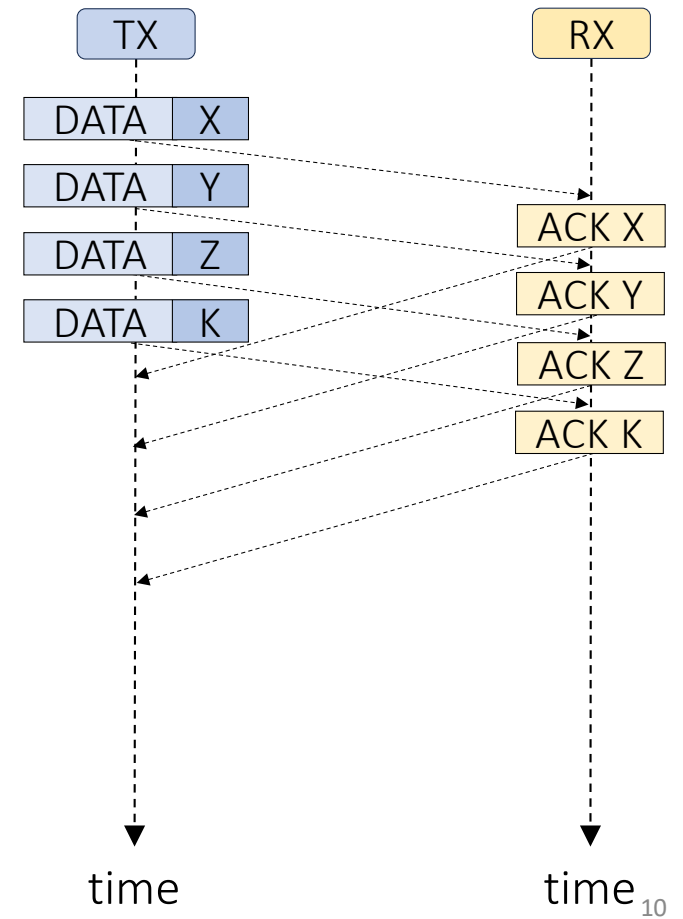
- Before data exchange the connection setup must happen
 - three way handshake: SYN, SYN-ACK, ACK
- The TCP stack in the blue server then splits the data in segments as network works with limited-sized packets
- It then send them over the network
- How many in-flight (without an ACK)?

This depends on the TCP window



Transmission Control Protocol (TCP) primer

- How to increase the TCP window (W)?



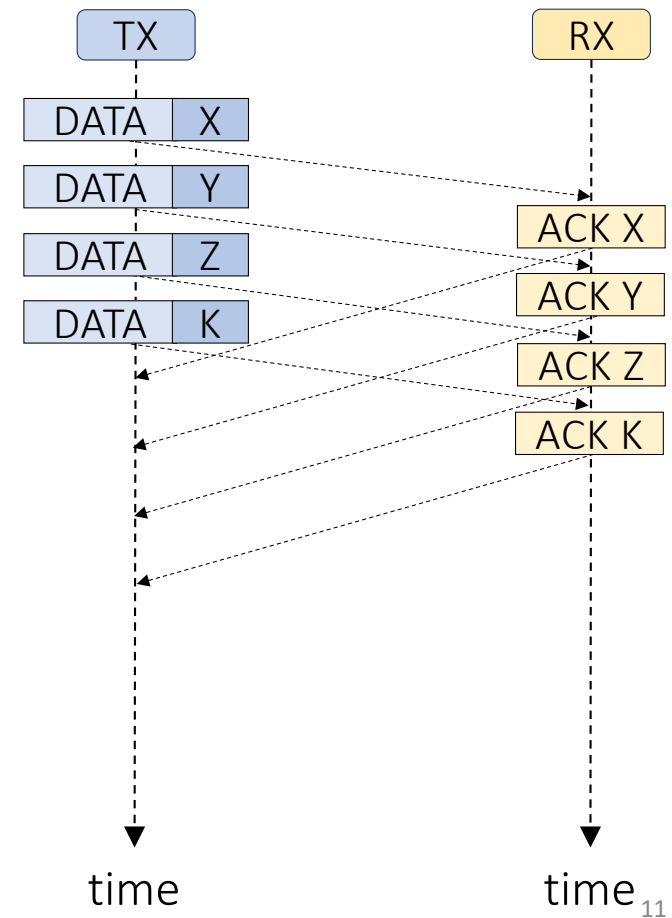
Transmission Control Protocol (TCP) primer

- How to increase the TCP window (W)?

at the reception of an ACK:

slow start: $W = W + 1$

congestion avoidance: $W = W + 1/W$



Transmission Control Protocol (TCP) primer

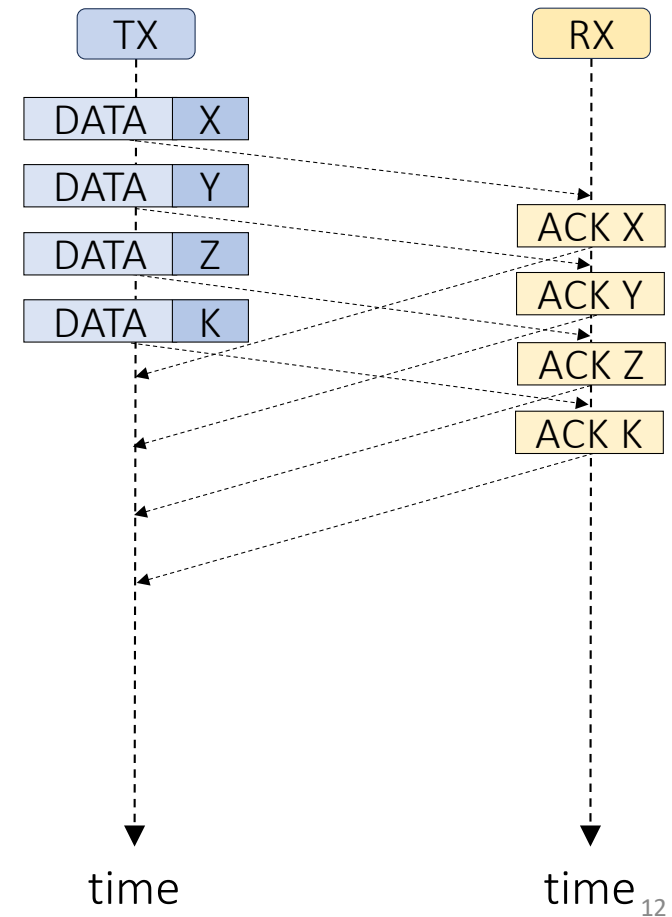
- How to increase the TCP window (W)?

at the reception of an ACK:

slow start: $W = W + 1$

congestion avoidance: $W = W + 1/W$

- When do you transition from *slow start* to *congestion avoidance*?



Transmission Control Protocol (TCP) primer

- How to increase the TCP window (W)?

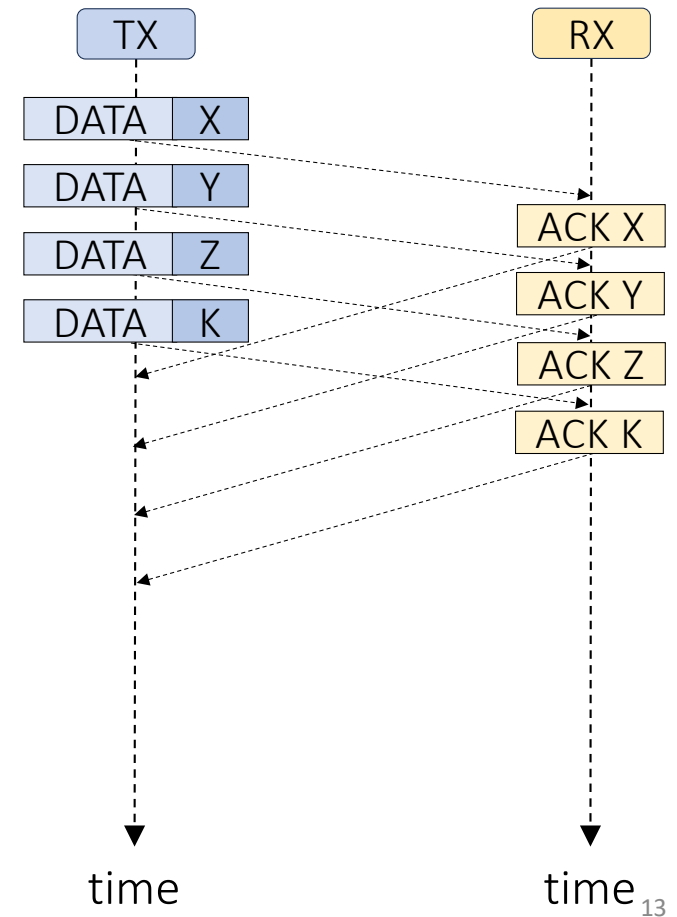
at the reception of an ACK:

slow start: $W = W + 1$

congestion avoidance: $W = W + 1/W$

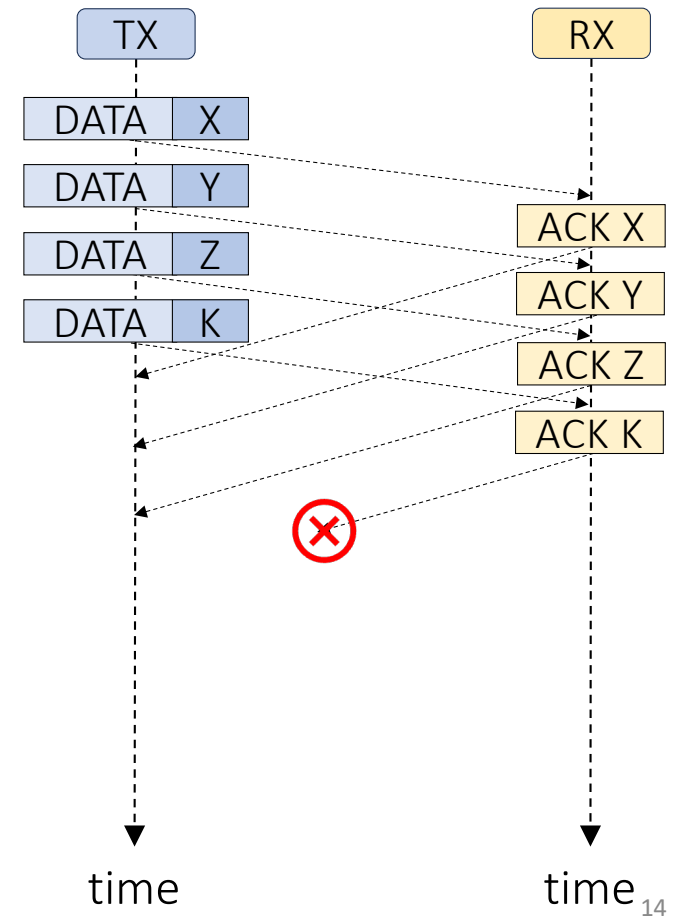
- When do you transition from *slow start* to *congestion avoidance*?

$W > SSTRESH$



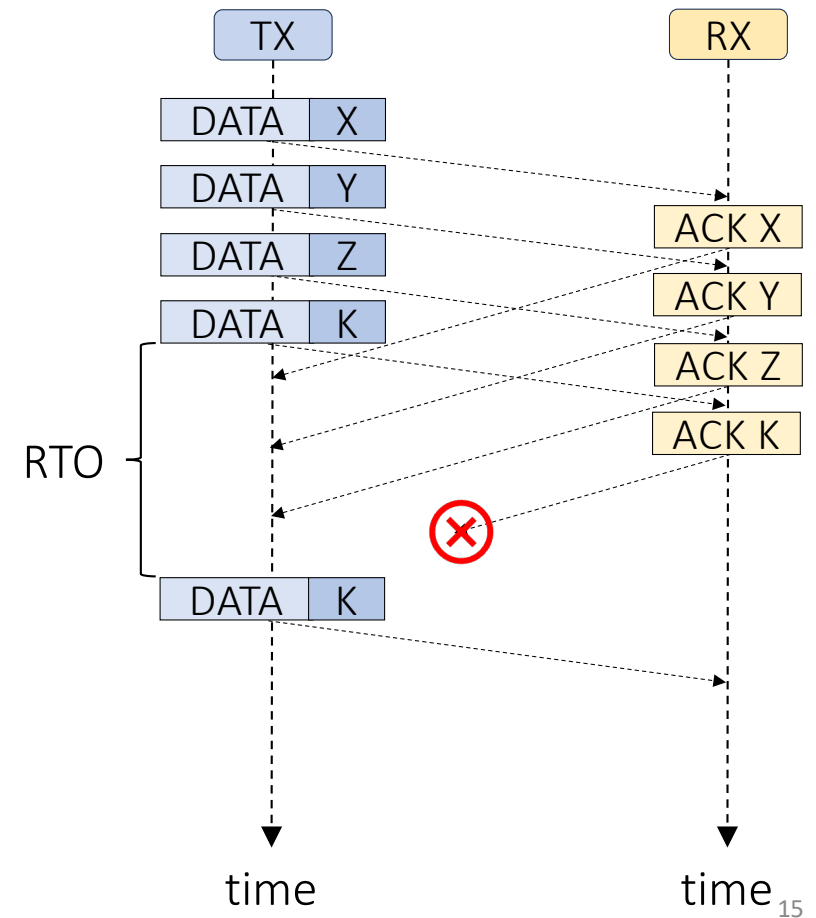
Transmission Control Protocol (TCP) primer

- ACKs can be lost



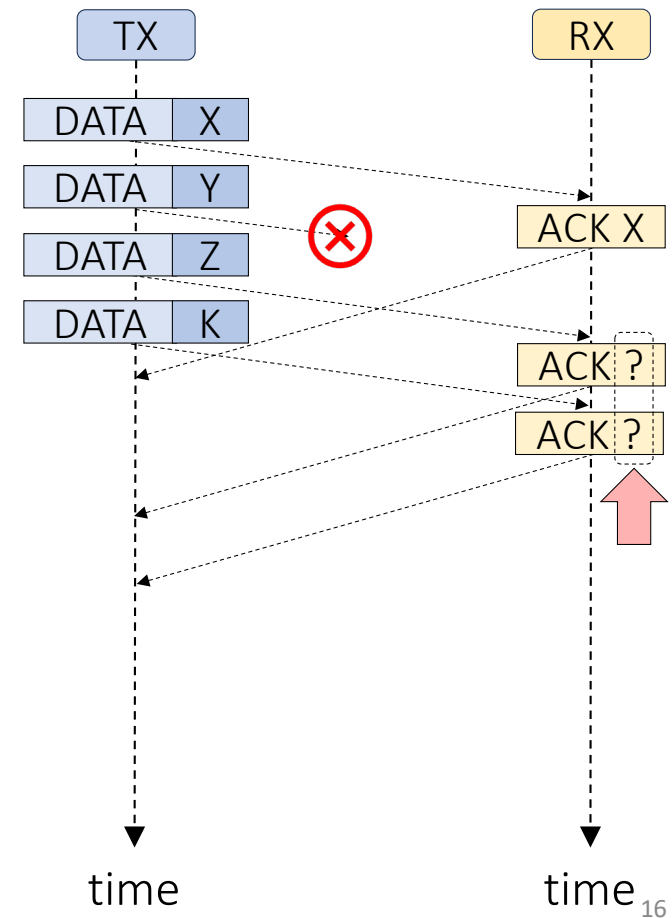
Transmission Control Protocol (TCP) primer

- ACKs can be lost
 - After a retransmission timeout (RTO) data is sent again and TCP Window decreased ☹



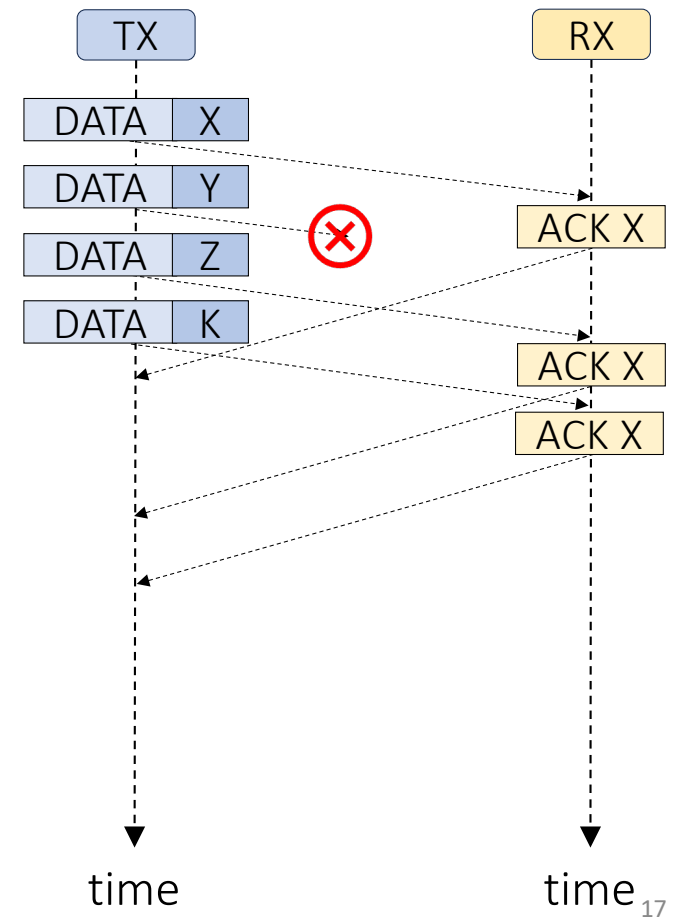
Transmission Control Protocol (TCP) primer

- ACKs can be lost
 - After a retransmission timeout (RTO) data is sent again and TCP Window decreased ☹
- Packets can be lost



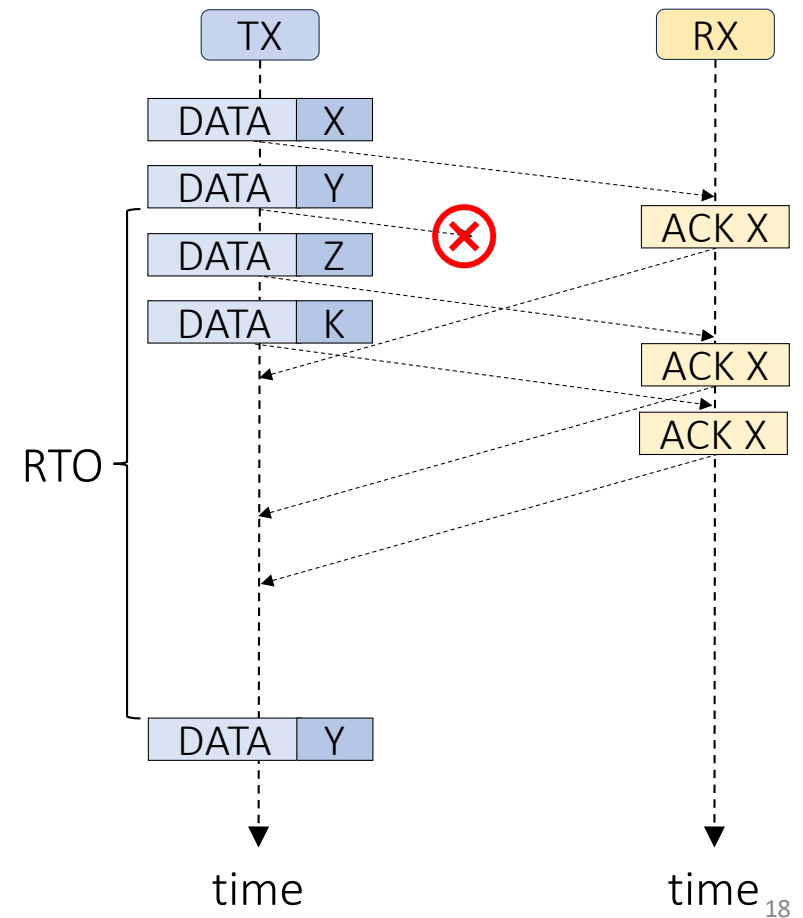
Transmission Control Protocol (TCP) primer

- ACKs can be lost
 - After a retransmission timeout (RTO) data is sent again and TCP Window decreased ☹
- Packets can be lost
 - ACKs following the lost packet carry the same sequence number



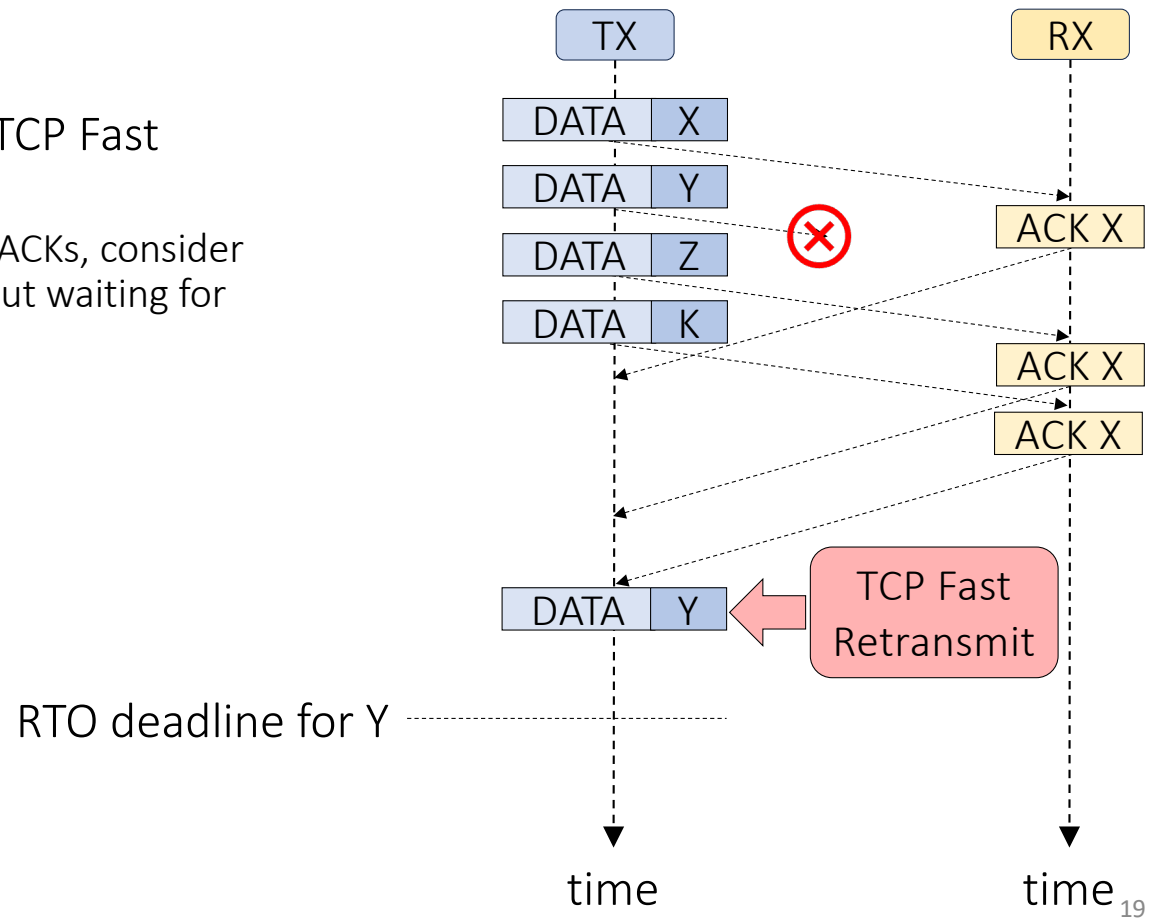
Transmission Control Protocol (TCP) primer

- ACKs can be lost
 - After a retransmission timeout (RTO) data is sent again and TCP Window decreased ☹
- Packets can be lost
 - ACKs following the lost packet carry the same sequence number and after RTO data is sent again and TCP Window decreased ☹



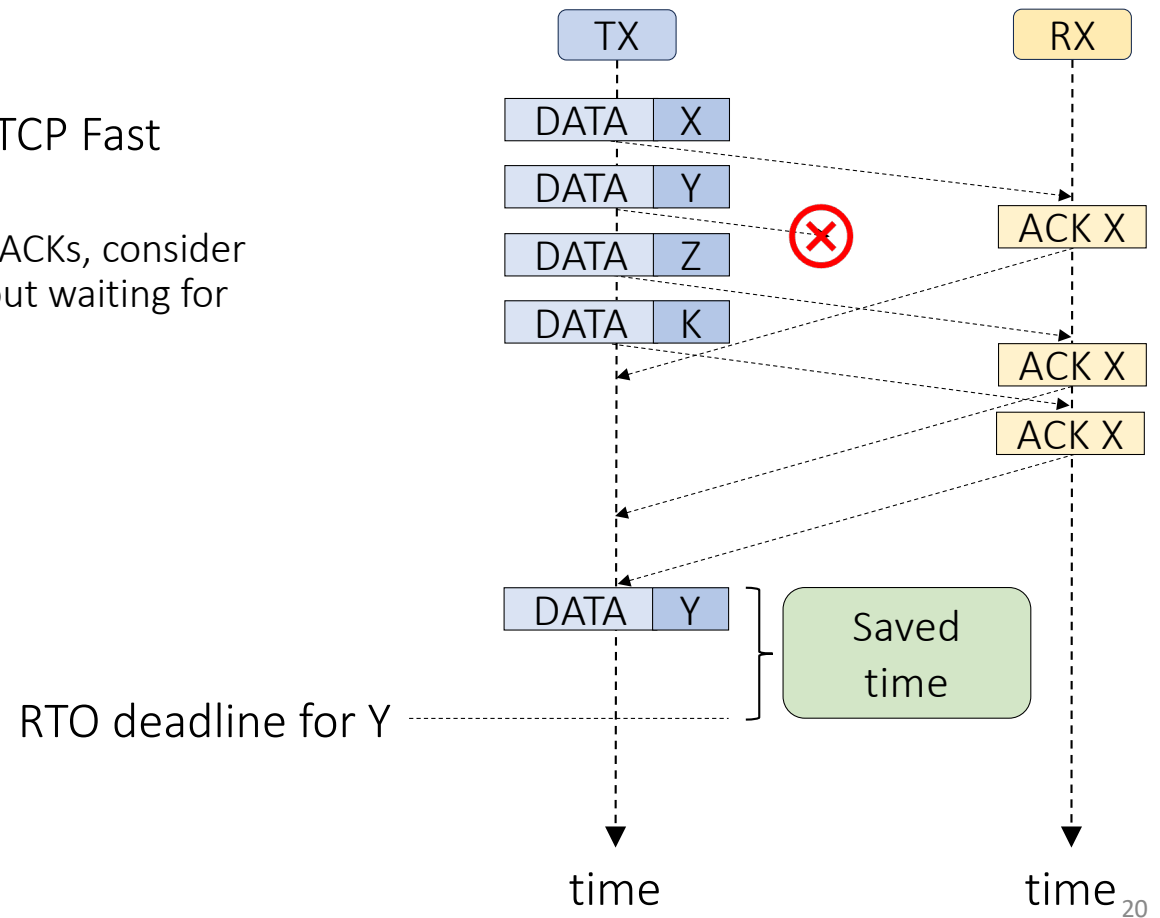
Transmission Control Protocol (TCP) primer

- We can improve performance with TCP Fast Retransmit:
 - when the sender sees three duplicate ACKs, consider the packet lost and send it again without waiting for RTO (still decrease TCP window ☹)



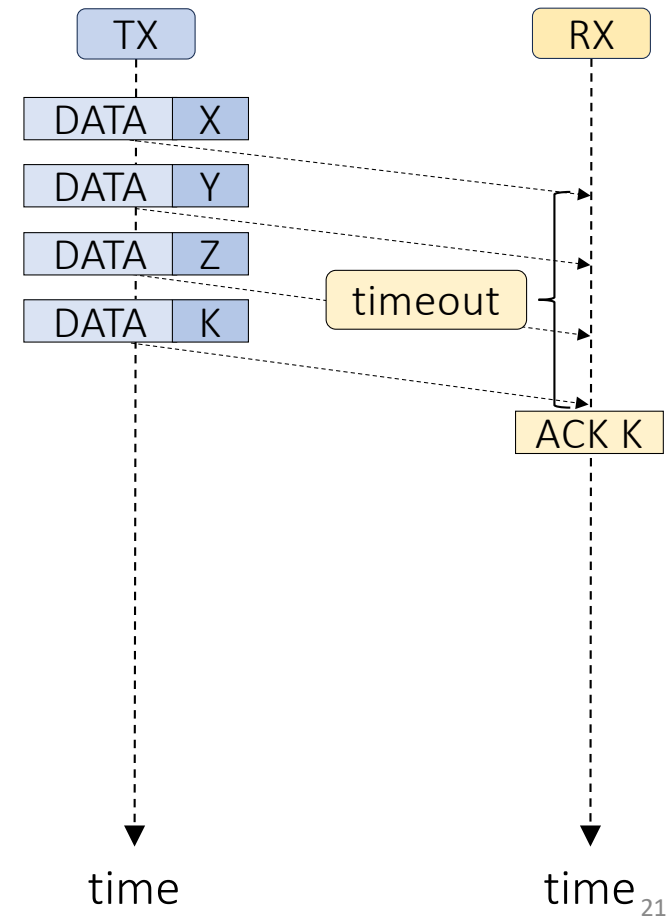
Transmission Control Protocol (TCP) primer

- We can improve performance with TCP Fast Retransmit:
 - when the sender sees three duplicate ACKs, consider the packet lost and send it again without waiting for RTO (still decrease TCP window ☹)



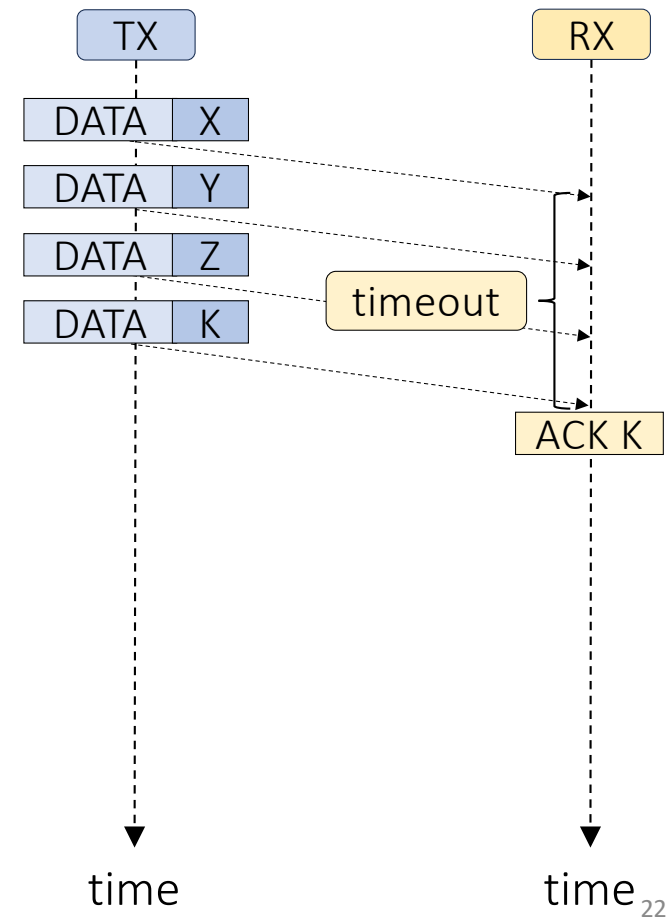
Transmission Control Protocol (TCP) primer

- To minimize the number of ACKs sent from receiver, we can delay them and send a cumulative ack after a timeout



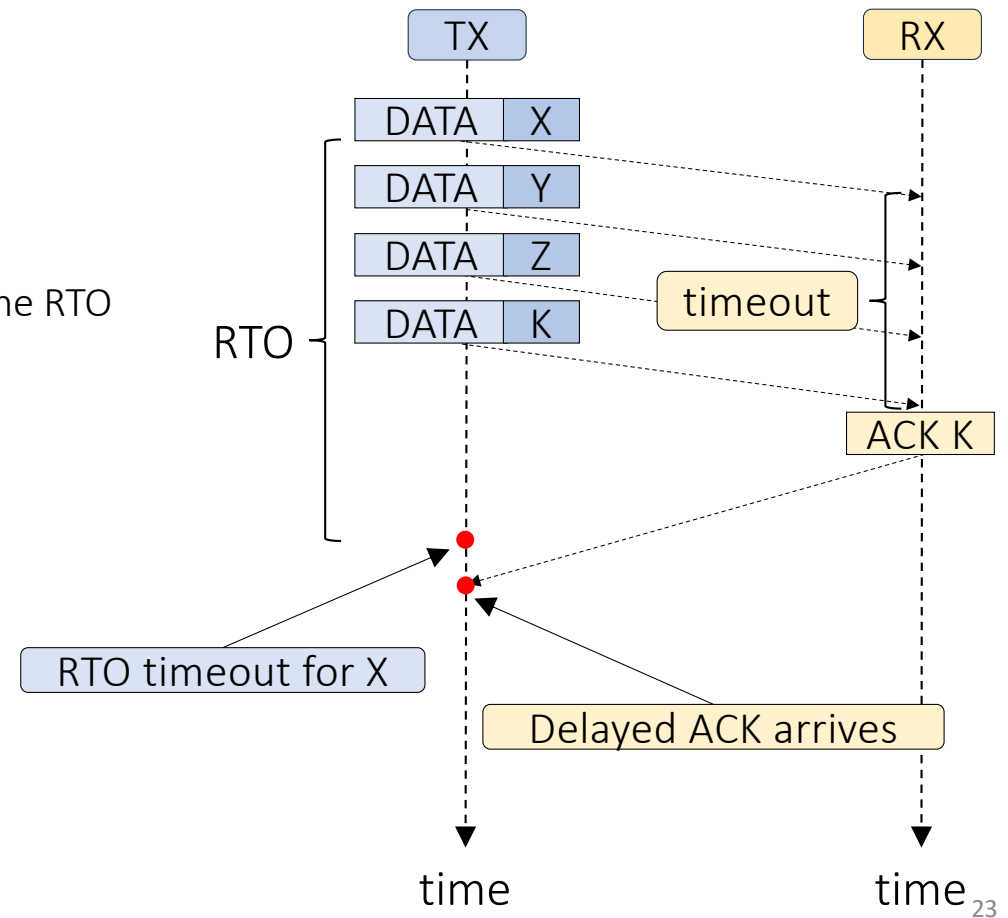
Transmission Control Protocol (TCP) primer

- To minimize the number of ACKs sent from receiver, we can delay them and send a cumulative ack after a timeout
 - What happens if the delayed ack arrives after the RTO timeout?



Transmission Control Protocol (TCP) primer

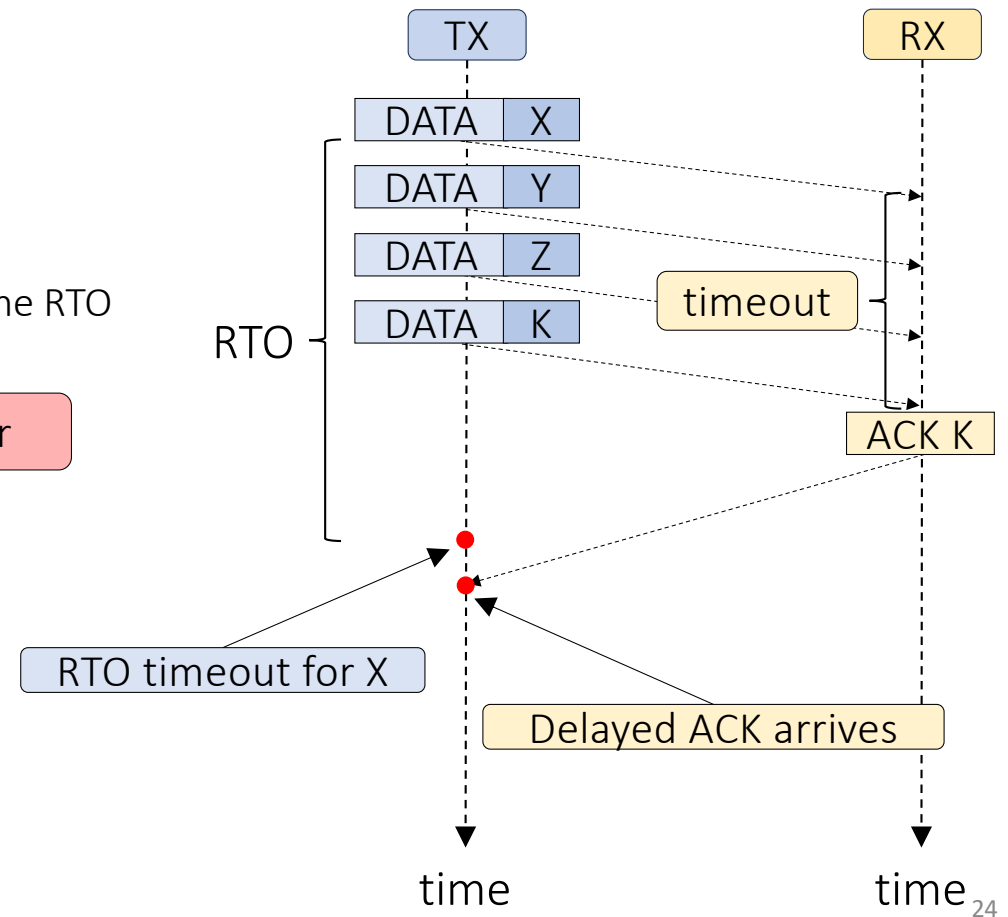
- To minimize the number of ACKs sent from receiver, we can delay them and send a cumulative ack after a timeout
 - What happens if the delayed ack arrives after the RTO timeout?



Transmission Control Protocol (TCP) primer

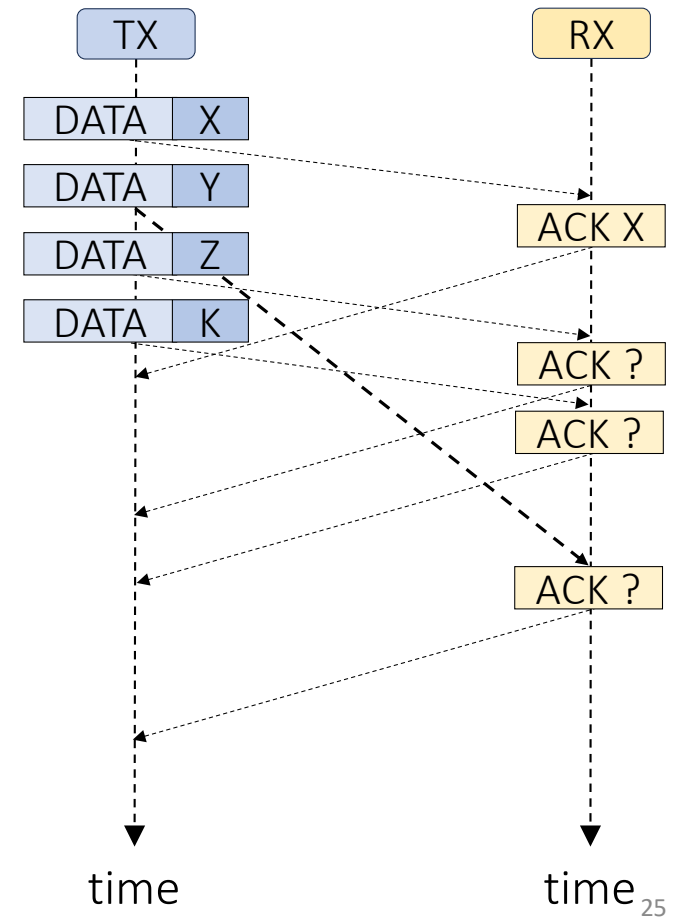
- To minimize the number of ACKs sent from receiver, we can delay them and send a cumulative ack after a timeout
 - What happens if the delayed ack arrives after the RTO timeout?

Packets will be treated as lost by the sender



Transmission Control Protocol (TCP) primer

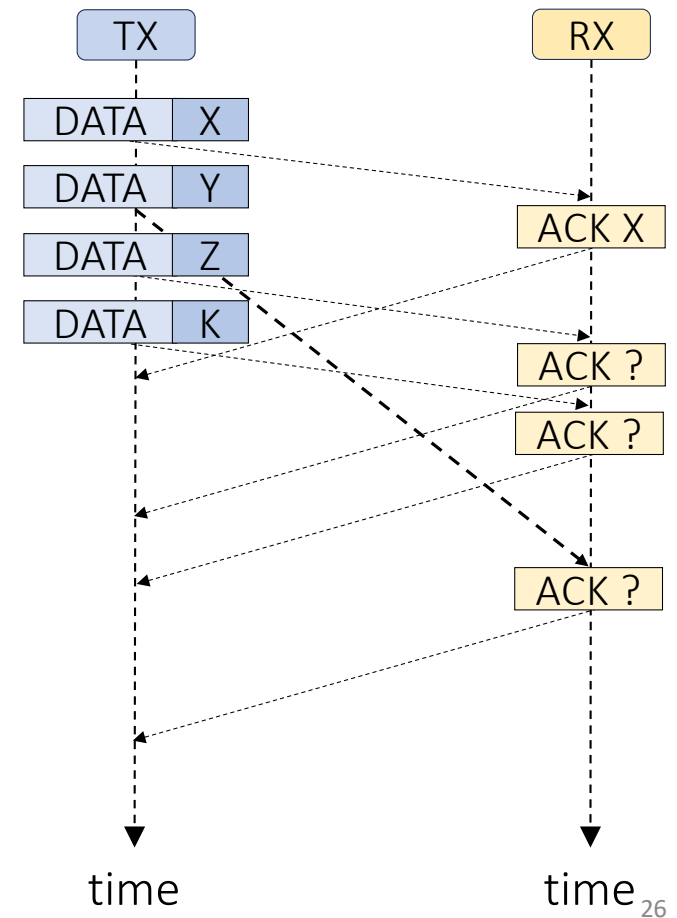
- Packets can be re-ordererd (why?)



Transmission Control Protocol (TCP) primer

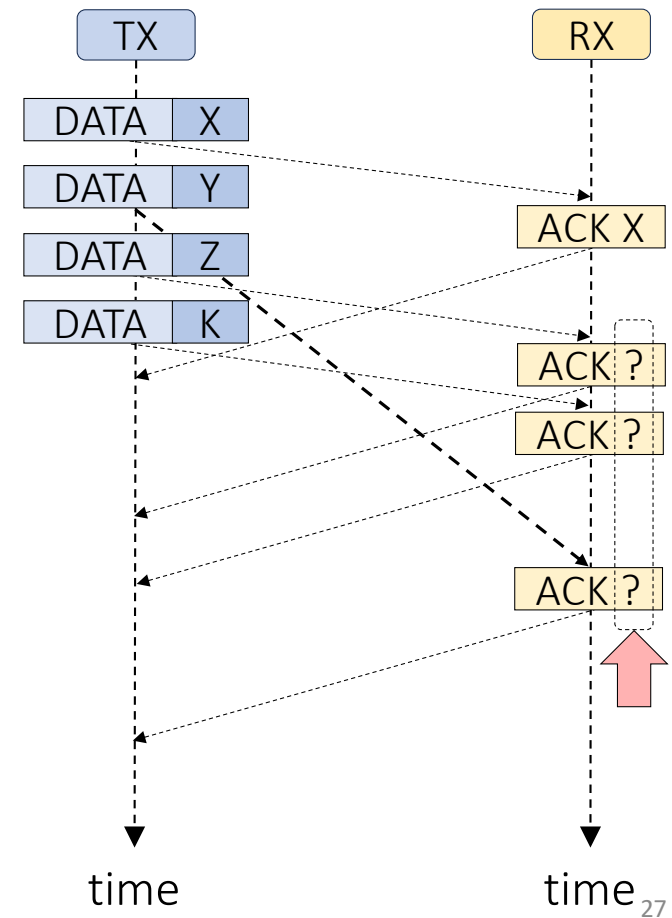
- Packets can be re-ordererd (why?)

Multipath can cause re-ordering



Transmission Control Protocol (TCP) primer

- Packets can be re-ordererd (why?)
Multipath can cause re-ordering
- How are ACKs handled in this scenario?



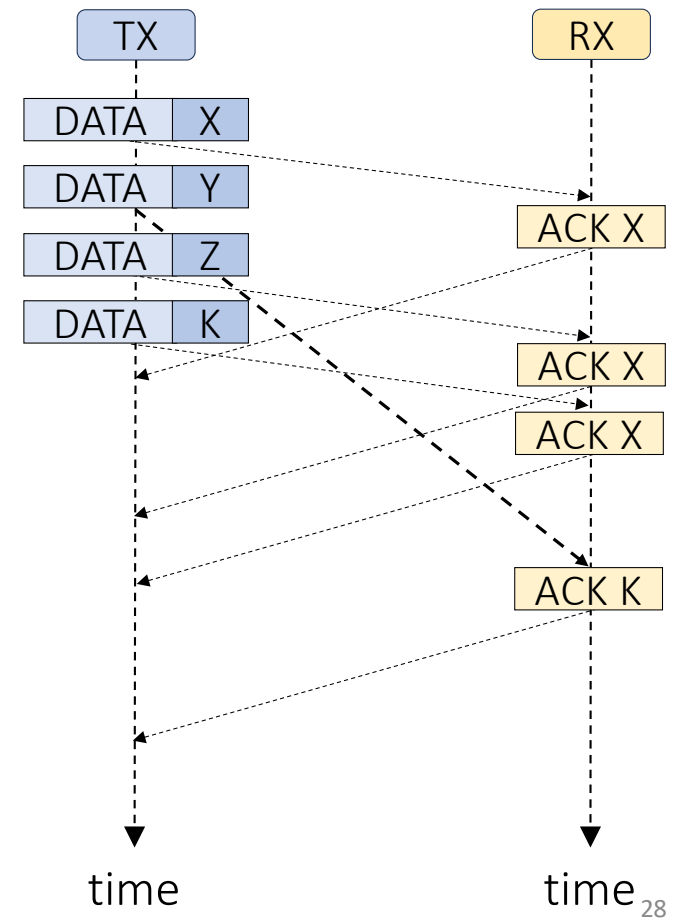
Transmission Control Protocol (TCP) primer

- Packets can be re-ordererd (why?)

Multipath can cause re-ordering

- How are ACKs handled in this scenario?

ACKs for out-of-order packets repeat the sequence number of the last in-order byte received



Transmission Control Protocol (TCP) primer

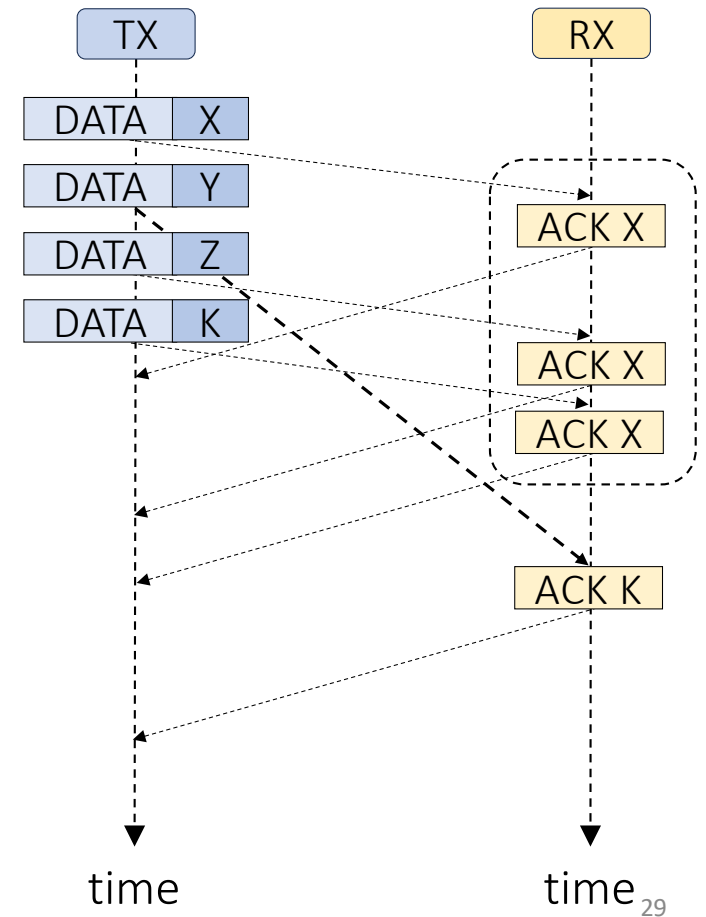
- Packets can be re-ordererd (why?)

Multipath can cause re-ordering

- How are ACKs handled in this scenario?

ACKs for out-of-order packets repeat the sequence number of the last in-order byte received

- What happens to fast retransmit?



Transmission Control Protocol (TCP) primer

- Packets can be re-ordererd (why?)

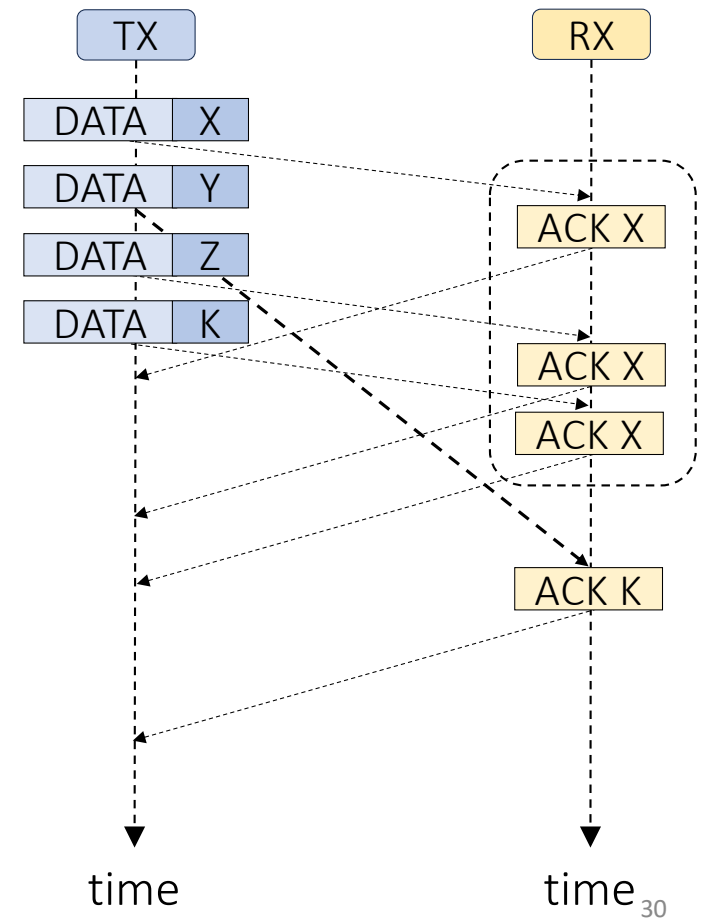
Multipath can cause re-ordering

- How are ACKs handled in this scenario?

ACKs for out-of-order packets repeat the sequence number of the last in-order byte received

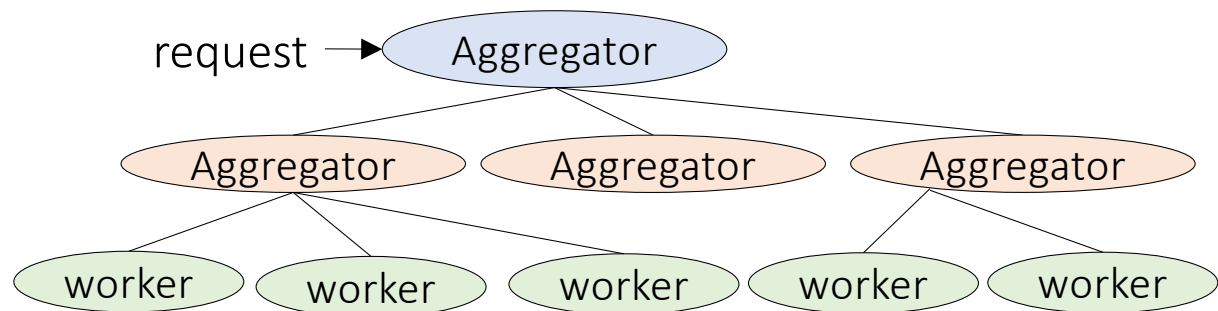
- What happens to fast retransmit?

TCP works badly in the presence of out-of-order packets



TCP and datacenter networking

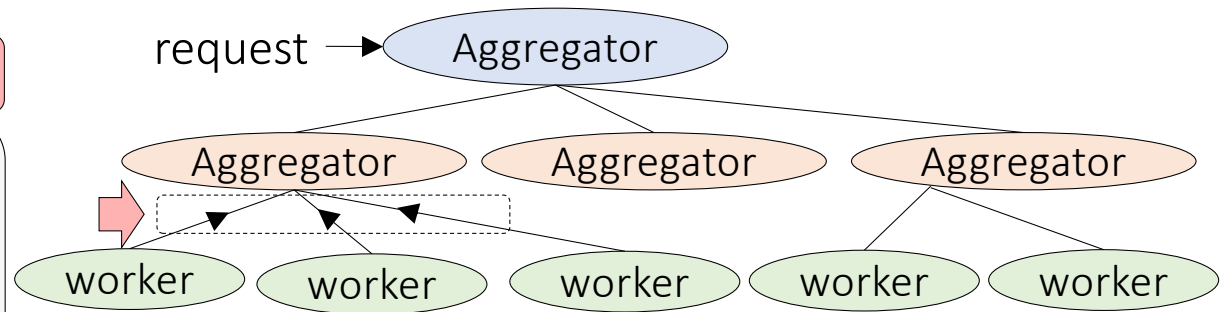
- TCP does not adapt that well to datacenter networking requirements
- Many large-scale applications follow the *partition/aggregate* design pattern, where work is divided across multiple workers and results are later combined
 - In a web search system, the index is partitioned across multiple machines, each searching a subset of documents. Upon a query, a front-end aggregator sends it to all workers in parallel; each returns partial results (e.g., top-ranked documents), which are then merged and ranked by the aggregator



TCP and datacenter networking

Impairment 1: Incast

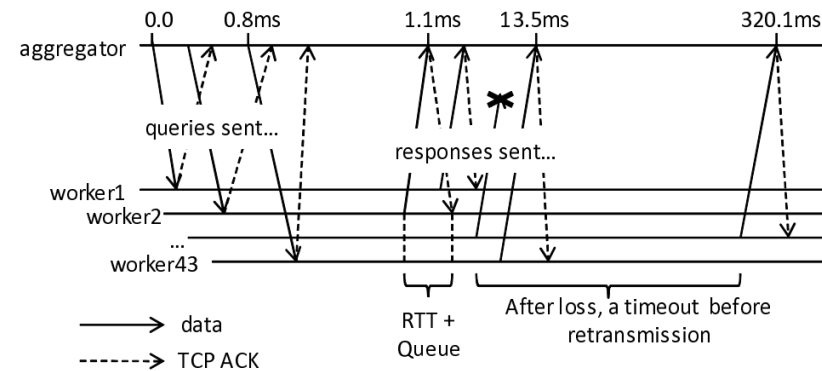
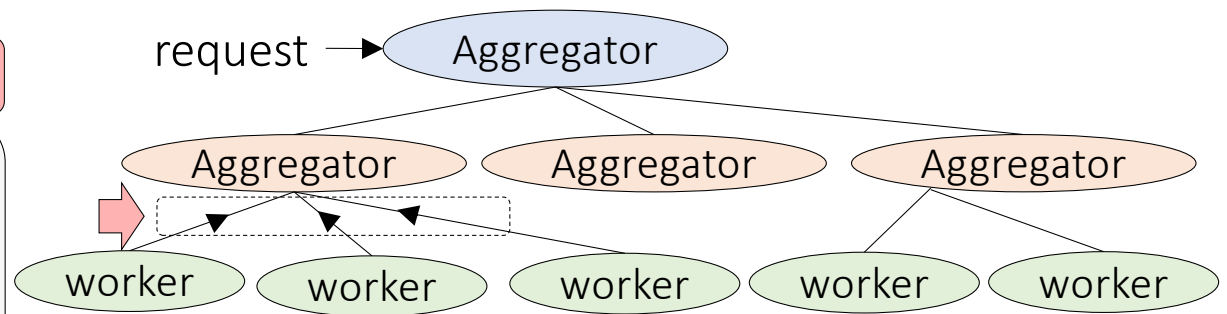
If many flows converge on the same switch interface over a short period of time, they might saturate the buffer and induce packet loss



TCP and datacenter networking

Impairment 1: Incast

If many flows converge on the same switch interface over a short period of time, they might saturate the buffer and induce packet loss



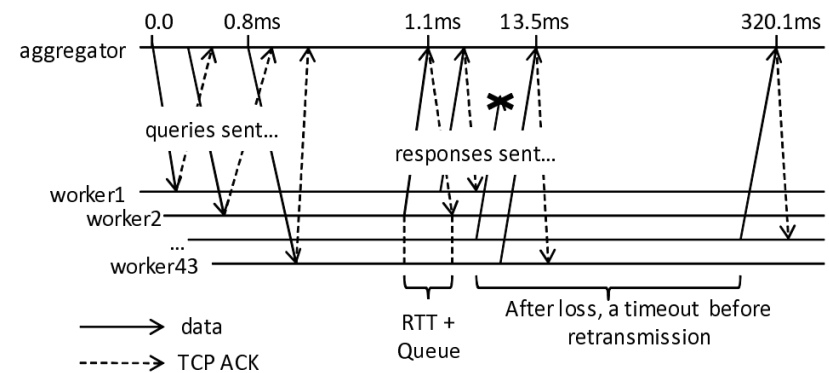
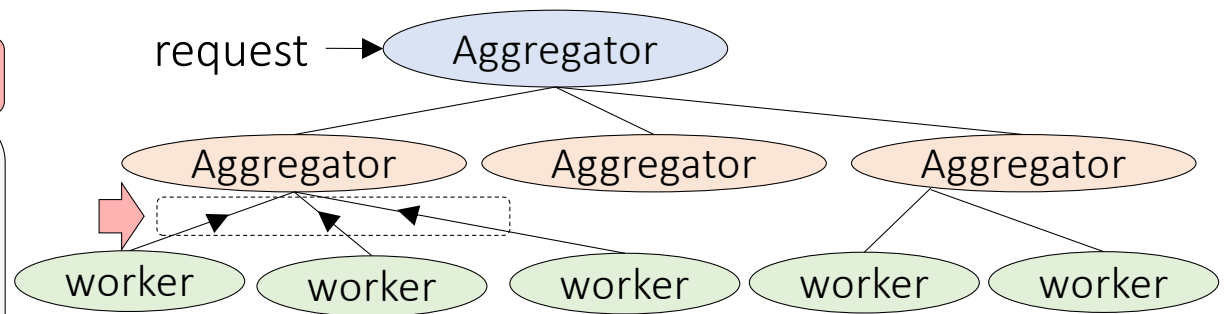
A real incast event measured in a production environment

TCP and datacenter networking

Impairment 1: Incast

If many flows converge on the same switch interface over a short period of time, they might saturate the buffer and induce packet loss

You can de-synchronise the workers delaying their answer randomly



A real incast event measured in a production environment

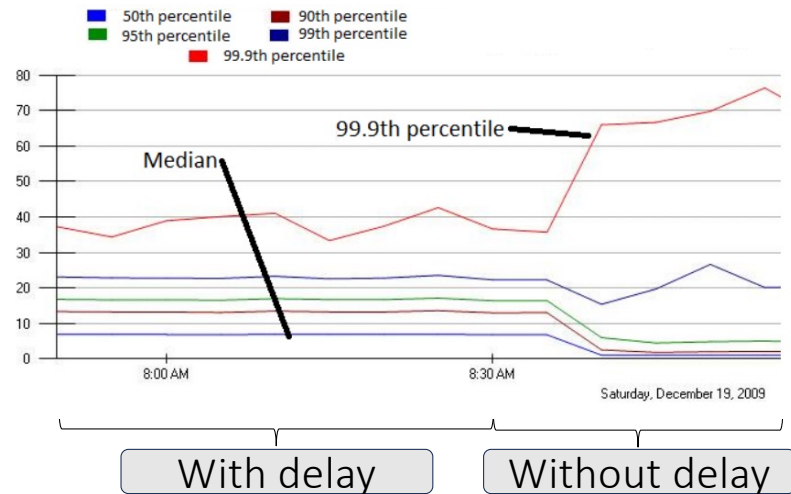
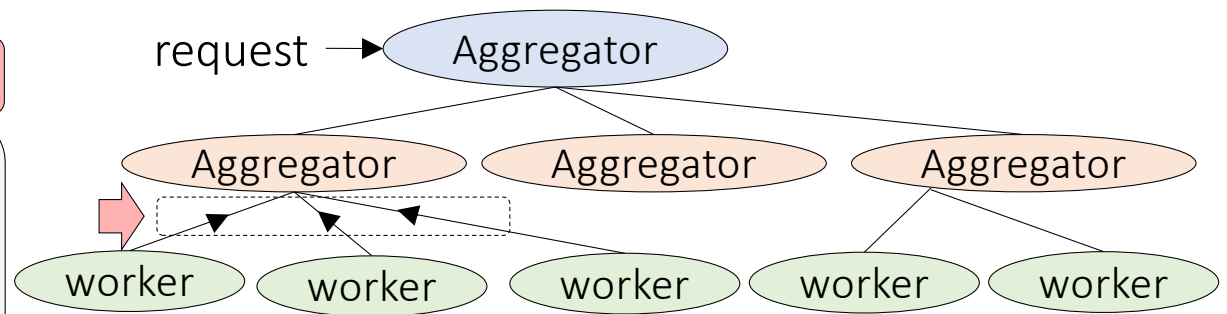
TCP and datacenter networking

Impairment 1: Incast

If many flows converge on the same switch interface over a short period of time, they might saturate the buffer and induce packet loss

You can de-synchronise the workers delaying their answer randomly

Jittering reduces tail latency at the cost of a higher median response time



TCP and datacenter networking

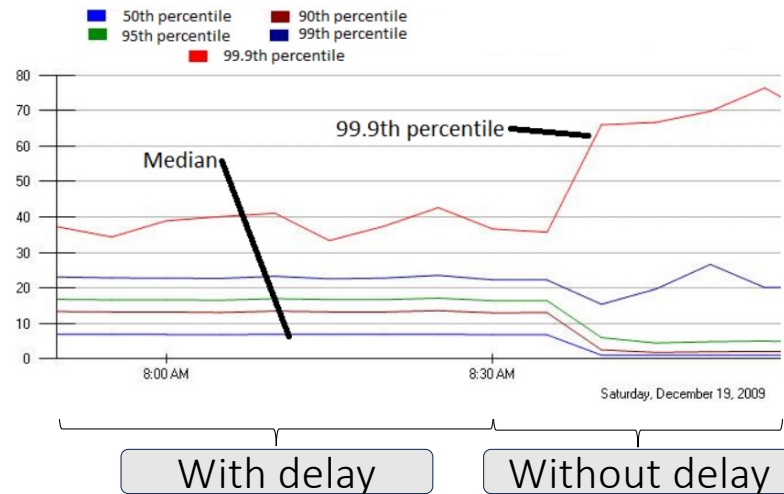
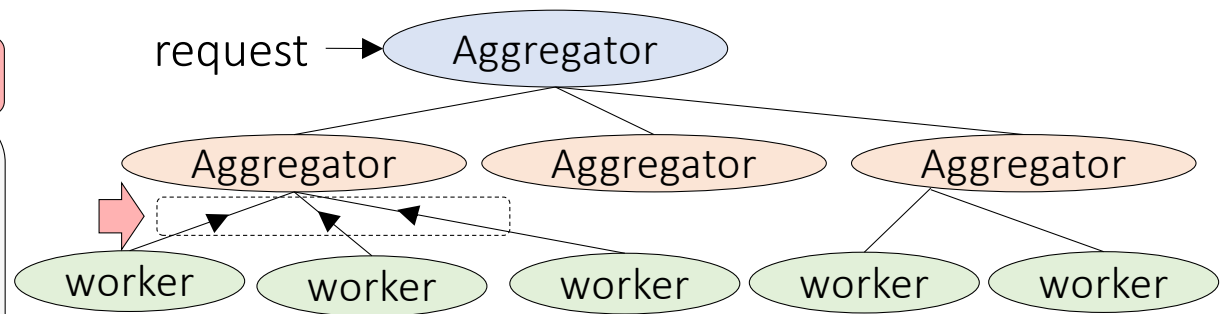
Impairment 1: Incast

If many flows converge on the same switch interface over a short period of time, they might saturate the buffer and induce packet loss

You can de-synchronise the workers delaying their answer randomly

Jittering reduces tail latency at the cost of a higher median response time

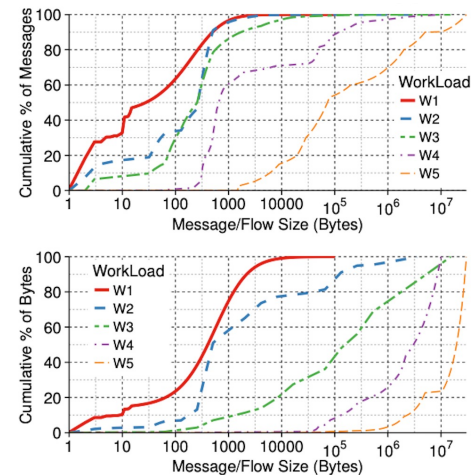
What if we reduce RTO?



TCP and datacenter networking

Impairment 2: Queue buildup

Every worker in the cluster handles both query traffic and background traffic

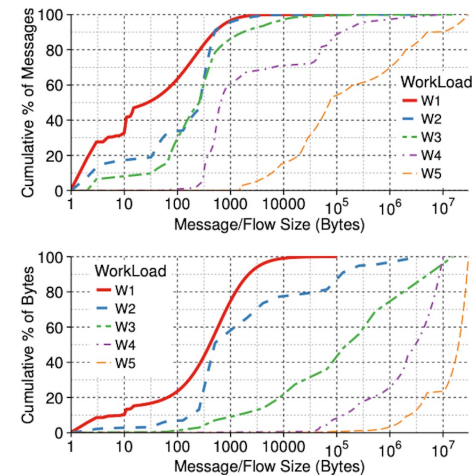


- W1 Accesses to a collection of memcached servers at Facebook, as approximated by the statistical model of the ETC workload in Section 5 and Table 5 of [5].
- W2 Search application at Google [29].
- W3 Aggregated workload from all applications running in a Google datacenter [29].
- W4 Hadoop cluster at Facebook [27].
- W5 Web search workload used for DCTCP [2].

TCP and datacenter networking

Impairment 2: Queue buildup

Every worker in the cluster handles both query traffic and background traffic



- W1 Accesses to a collection of memcached servers at Facebook, as approximated by the statistical model of the ETC workload in Section 5 and Table 5 of [5].
- W2 Search application at Google [29].
- W3 Aggregated workload from all applications running in a Google datacenter [29].
- W4 Hadoop cluster at Facebook [27].
- W5 Web search workload used for DCTCP [2].



Short messages



low latency



Large flows



high throughput

TCP and datacenter networking

Impairment 2: Queue buildup

Every worker in the cluster handles both query traffic and background traffic



Short messages



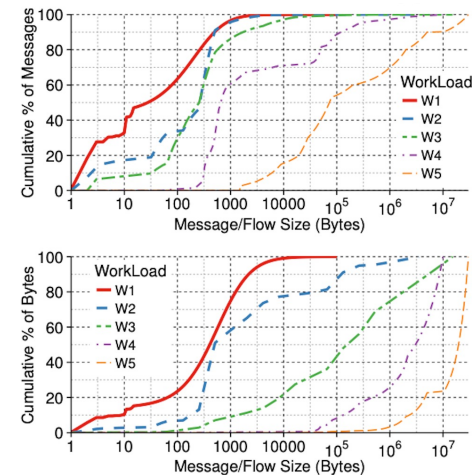
low latency



Large flows



high throughput

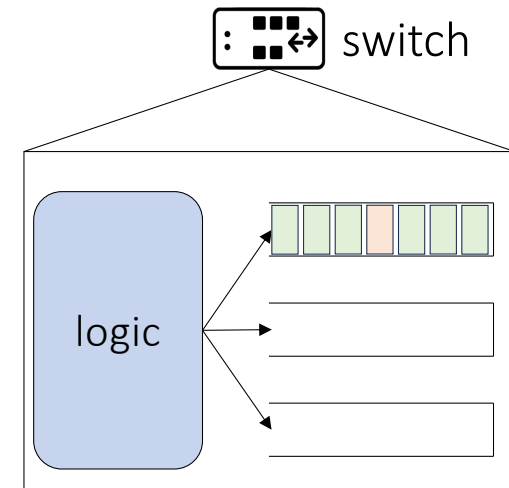


- W1 Accesses to a collection of memcached servers at Facebook, as approximated by the statistical model of the ETC workload in Section 5 and Table 5 of [5].
- W2 Search application at Google [29].
- W3 Aggregated workload from all applications running in a Google datacenter [29].
- W4 Hadoop cluster at Facebook [27].
- W5 Web search workload used for DCTCP [2].

Consequence: short and long flows compete for space in the output queue buffer of a switch

TCP and datacenter networking

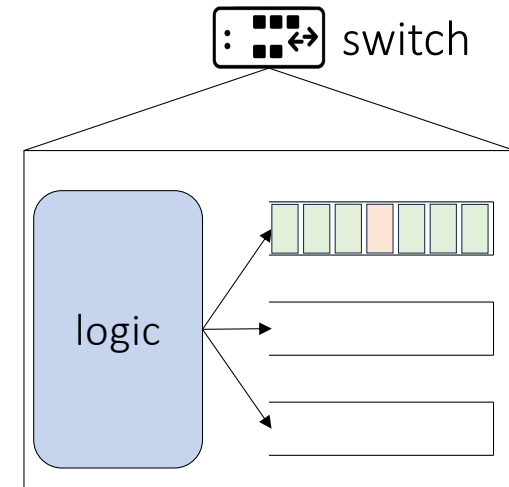
- Two outcomes:
 - Packet loss (incast problem)
 - Queue buildup (short flows experience increased latency as they are in queue behind packets from large flows)



TCP and datacenter networking

- Two outcomes:
 - Packet loss (incast problem)
 - Queue buildup (short flows experience increased latency as they are in queue behind packets from large flows)

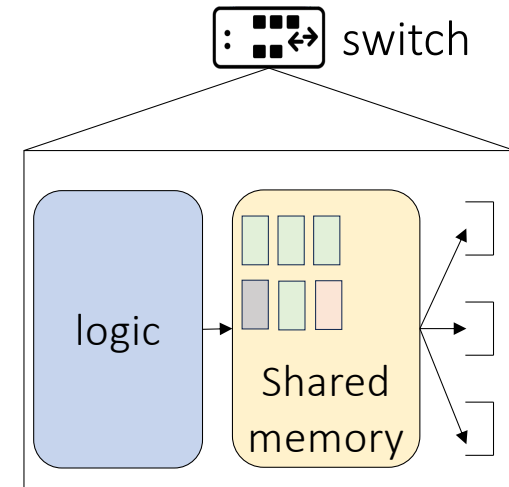
Reducing RTO does not help ☹️



TCP and datacenter networking

Impairment 3: Buffer pressure

- Buffer at switches is commonly shared between ports
- Long TCP flows build up queues and since buffer space is a shared resource, there is less space for other flows

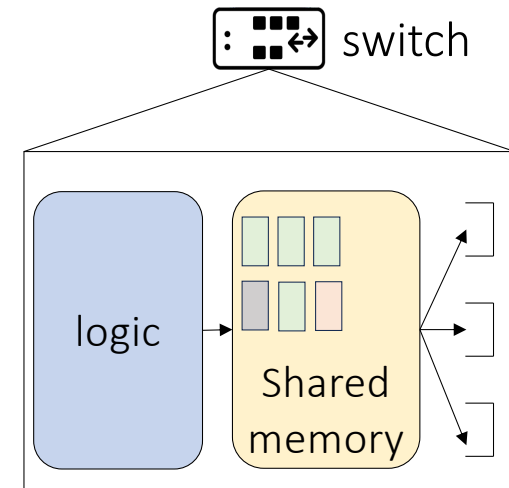


TCP and datacenter networking

Impairment 3: Buffer pressure

- Buffer at switches is commonly shared between ports
- Long TCP flows build up queues and since buffer space is a shared resource, there is less space for other flows

Consequence: the loss rate of short flows depends on the number of long flows traversing other ports



Datacenter transport requirements

- High burst tolerance
 - Incast due to partition/aggregate is common
- Low latency
 - Short flows, queries
- High throughput
 - Continuous data updates, large file transfers

Datacenter transport requirements

- High burst tolerance
 - Incast due to partition/aggregate is common
- Low latency
 - Short flows, queries
- High throughput
 - Continuous data updates, large file transfers

The challenge is to achieve these three together!

Datacenter transport requirements

- High burst tolerance
 - Incast due to partition/aggregate is common
- Low latency
 - Short flows, queries
- High throughput
 - Continuous data updates, large file transfers

The challenge is to achieve these three together!

A thought example:

To guarantee high burst tolerance and high throughput I might be inclined to use deep buffers

Datacenter transport requirements

- High burst tolerance
 - Incast due to partition/aggregate is common
- Low latency
 - Short flows, queries
- High throughput
 - Continuous data updates, large file transfers

The challenge is to achieve these three together!

A thought example:

To guarantee high burst tolerance and high throughput I might be inclined to use deep buffers

Is this good for latency?

Datacenter transport requirements

- High burst tolerance
 - Incast due to partition/aggregate is common
- Low latency
 - Short flows, queries
- High throughput
 - Continuous data updates, large file transfers

The challenge is to achieve these three together!

A thought example:

To guarantee high burst tolerance and high throughput I might be inclined to use deep buffers

Is this good for latency?

Deep buffers induce higher queuing latency ☹

Datacenter transport requirements

- High burst tolerance
 - Incast due to partition/aggregate is common
- Low latency
 - Short flows, queries
- High throughput
 - Continuous data updates, large file transfers

The challenge is to achieve these three together!

A thought example:

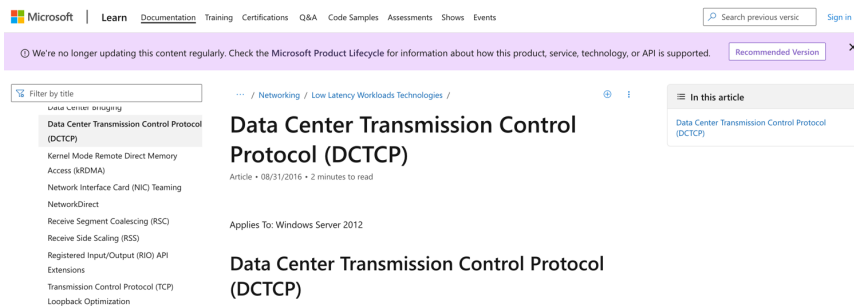
To guarantee high burst tolerance and high throughput I might be inclined to use deep buffers

Is this good for latency?

Deep buffers induce higher queuing latency ☹

Router buffers are traditionally sized using the Bandwidth Delay Product rule ($RTT \times C$).
It has been demonstrated that $C \times RTT / \sqrt{N}$ is enough!

Datacenter TCP (DCTCP)



Data Center TCP (DCTCP)

Mohammad Alizadeh^{††}, Albert Greenberg[†], David A. Maltz[†], Jitendra Padhye[†],
Parveen Patel[†], Balaji Prabhakar[†], Sudipta Sengupta[†], Murari Sridharan[†]

[†]Microsoft Research [†]Stanford University
{albert, dmaltz, padhye, parveenp, sudipta, muraris}@microsoft.com
{alizade, balaji}@stanford.edu

- A solution proposed by Microsoft and running in production in their datacenters
- Built to work with legacy switches on top of the commodity ECN feature
- Work presented at ACM SIGCOMM 2010 (New Delhi)

Datacenter TCP (DCTCP)

A large-scale deployment of DCTCP

Operational Systems Track

- ...not only Microsoft

Abhishek Dhamija,¹ Balasubramanian Madhavan,¹ Hechao Li,² Jie Meng,¹ Shrikrishna Khare,¹ Madhavi Rao,¹ Lawrence Brakmo, Neil Spring,¹ Prashanth Kannan,¹ Srikanth Sundaresan,¹ and Soudeh Ghorbani^{1,3}

¹Meta, ²Netflix, ³Johns Hopkins University

Abstract

This paper describes the process and operational experiences of deploying the Data Center TCP (DCTCP) protocol in a large-scale data center network. In contrast to legacy congestion control protocols that rely on loss as the primary signal of congestion, DCTCP signals in-network congestion (based on queue occupancy) to senders and adjusts the sending rate proportional to the level of congestion. At the time of our deployment, this protocol was well-studied and fairly established with proven efficiency gains in other networks. As expected, we also observed improved performance, and notably decreased packet losses, compared to legacy protocols in our data centers. Perhaps unexpectedly, however, we faced numerous hurdles in rolling out DCTCP; we chronicle these unexpected challenges, ranging from its unfairness (to other classes of traffic) to implementation bugs. We close by discussing some of the open research questions and challenges.

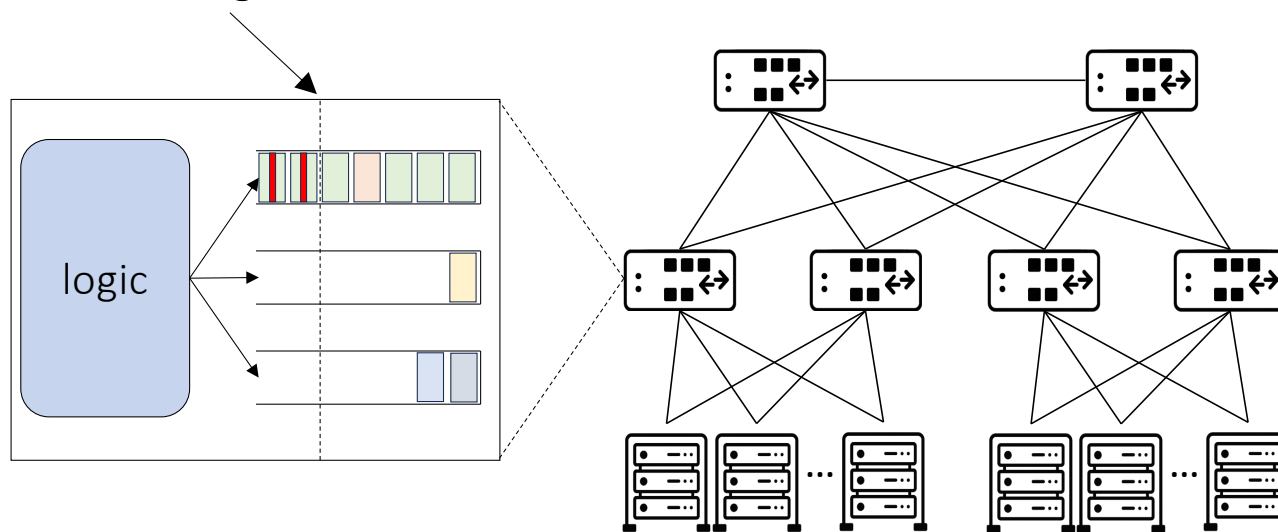
a diverse set of congestion signals (notifications from the network, measured delay at endpoints, etc.) to detect and react to imminent congestion faster. As one of the earliest and most mature protocols in this class, Data Center Congestion Control (DCTCP) [4] uses Explicit Congestion Notifications (ECNs) from the switches to adjust the sending rate proportional to the level of congestion. ECN remedies large queue buildups and drops by providing a congestion signal before queues overflow, and DCTCP interprets the fraction of ECN-marked packets to scale its response, avoiding persistent queueing and overflows that lead to loss.

A few features of our network made DCTCP's potential worth considering. First, our top of rack switches had "shallow buffers," providing limited space for queueing. Second, buffer sharing and contention over buffers led to variable and unpredictable queue capacities [13]. Finally, our approach to distributing jobs across datacenters created a set of racks with a mixture of large throughput-heavy flows and small

Explicit Congestion Notification

- Explicit Congestion Notification (ECN) is a TCP/IP extension (RFC 3168)
- ECN allows end-to-end notification of network congestion without dropping packets

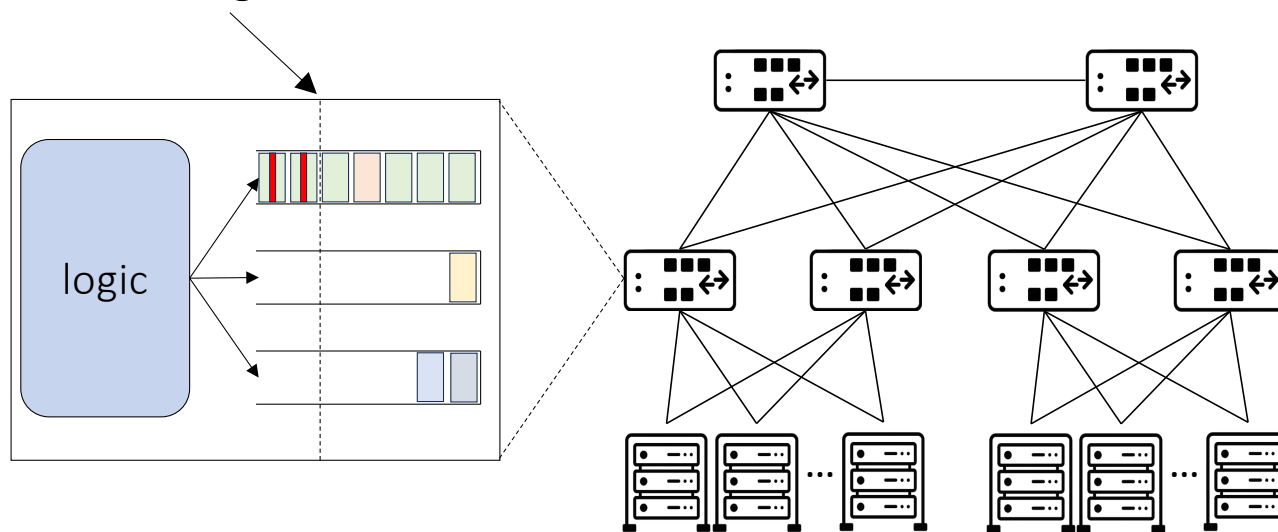
ECN marking threshold



Explicit Congestion Notification

- Explicit Congestion Notification (ECN) is a TCP/IP extension (RFC 3168)
- ECN allows end-to-end notification of network congestion without dropping packets

ECN marking threshold



ECN marking in routers with shared memory buffers follows the same principles, but the marking decision is based on shared-buffer occupancy rather than a single queue's depth

Explicit Congestion Notification

00: Not ECN-Capable
01: ECN Capable Transport
10: ECN Capable Transport
11: Congestion Experienced

- ECN requires support at the IP layer

Ver.	IHL	DSCP	ECN	Total Length	
IPID				Flags	Fragment Offest
TTL		Protocol		Header Checksum	
Source IP					
Destination IP					

Explicit Congestion Notification

00: Not ECN-Capable
01: ECN Capable Transport
10: ECN Capable Transport
11: Congestion Experienced

- ECN requires support at the IP layer

Ver.	IHL	DSCP	ECN	Total Length	
IPID				Flags	Fragment Offest
TTL		Protocol		Header Checksum	
Source IP					
Destination IP					

- When a packet with CE set arrives at the receiver
 - The IP layer passes this information up to the TCP layer
 - The TCP implementation in the OS kernel records that congestion was experienced
 - The receiver's TCP stack signals congestion to the sender by setting the ECE flag in the TCP header of ACKs

Explicit Congestion Notification

00: Not ECN-Capable
01: ECN Capable Transport
10: ECN Capable Transport
11: Congestion Experienced

- ECN requires support at the IP layer

Ver.	IHL	DSCP	ECN	Total Length	
IPID				Flags	Fragment Offest
TTL		Protocol		Header Checksum	
Source IP					
Destination IP					

- When a packet with CE set arrives at the receiver
 - The IP layer passes this information up to the TCP layer
 - The TCP implementation in the OS kernel records that congestion was experienced
 - The receiver's TCP stack signals congestion to the sender by setting the ECE flag in the TCP header of ACKs

ECN flags are present both in the IP and TCP headers

The main idea behind DCTCP

- Extract multi-bit feedback from single-bit stream of ECN marks
 - Reduce window size based on *fraction* of marked packets
- The idea is to react in proportion to the extent of the congestion

ECN Marks	TCP	DCTCP
1 0 1 1 1 1 0 1 1 1	Cut window by 50%	Cut window by 40%
0 0 0 0 0 0 0 0 0 1	Cut window by 50%	Cut window by 5%

The DCTCP receiver

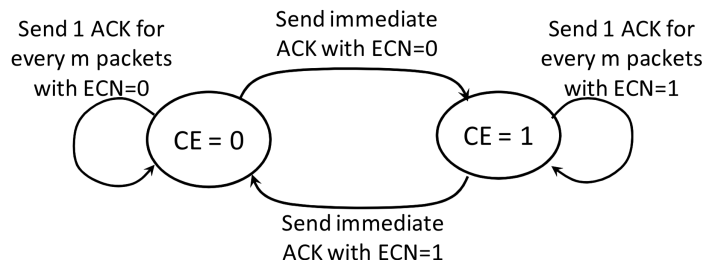
- The only difference between TCP and DCTCP is the way information in the CE codepoints is conveyed back to the sender
- DCTCP tries to accurately convey the exact sequence of marked packets back to the sender using only 1-bit CE information

How to do that without ACKing every packet that is suboptimal?

The DCTCP receiver

- The only difference between TCP and DCTCP is the way information in the CE codepoints is conveyed back to the sender
- DCTCP tries to accurately convey the exact sequence of marked packets back to the sender using only 1-bit CE information

How to do that without ACKing every packet that is suboptimal?



Use a novel two state ACK generation state machine

The DCTCP sender

- TCP always cut its window size by a factor of two in response to a marked ACK
- DCTCP starts reducing its window gently: $\mathbf{cwnd} \leftarrow (1 - \frac{\alpha}{2})\mathbf{cwnd}$

where.. $\alpha \leftarrow (1 - g)\alpha + gF$ and $F = \frac{\# \text{ of marked ACKs}}{\text{Total \# of ACKs}}$

ECN Marks	TCP	DCTCP
1 0 1 1 1 1 0 1 1 1	Cut window by 50%	Cut window by 40%
0 0 0 0 0 0 0 0 0 1	Cut window by 50%	Cut window by 5%

The DCTCP sender

- TCP always cut its window size by a factor of two in response to a marked ACK

- DCTCP starts reducing its window gently: $cwnd \leftarrow (1 - \frac{\alpha}{2})cwnd$

This estimate the extent of the congestion

where.. $\alpha \leftarrow (1 - g)\alpha + gF$ and

$$F = \frac{\# \text{ of marked ACKs}}{\text{Total \# of ACKs}}$$

ECN Marks	TCP	DCTCP
1 0 1 1 1 1 0 1 1 1	Cut window by 50%	Cut window by 40%
0 0 0 0 0 0 0 0 0 1	Cut window by 50%	Cut window by 5%

The DCTCP sender

- TCP always cut its window size by a factor of two in response to a marked ACK

- DCTCP starts reducing its window gently: $cwnd \leftarrow (1 - \frac{\alpha}{2})cwnd$

This estimate the extent of the congestion

where.. $\alpha \leftarrow (1 - g)\alpha + gF$ and $F = \frac{\# \text{ of marked ACKs}}{\text{Total \# of ACKs}}$

g is the weight given to new samples

ECN Marks	TCP	DCTCP
1 0 1 1 1 1 0 1 1 1	Cut window by 50%	Cut window by 40%
0 0 0 0 0 0 0 0 0 1	Cut window by 50%	Cut window by 5%

The DCTCP sender

when a is near 1 (high congestion), DCTCP behaves like TCP
when a is near 0 (low congestion), DCTCP gently reduces the window

- TCP always cut its window size by a factor of two in response to a marked ACK
- DCTCP starts reducing its window gently:

$$cwnd \leftarrow (1 - \frac{\alpha}{2})cwnd$$

This estimate the extent of the congestion

where.. $\alpha \leftarrow (1 - g)\alpha + gF$ and

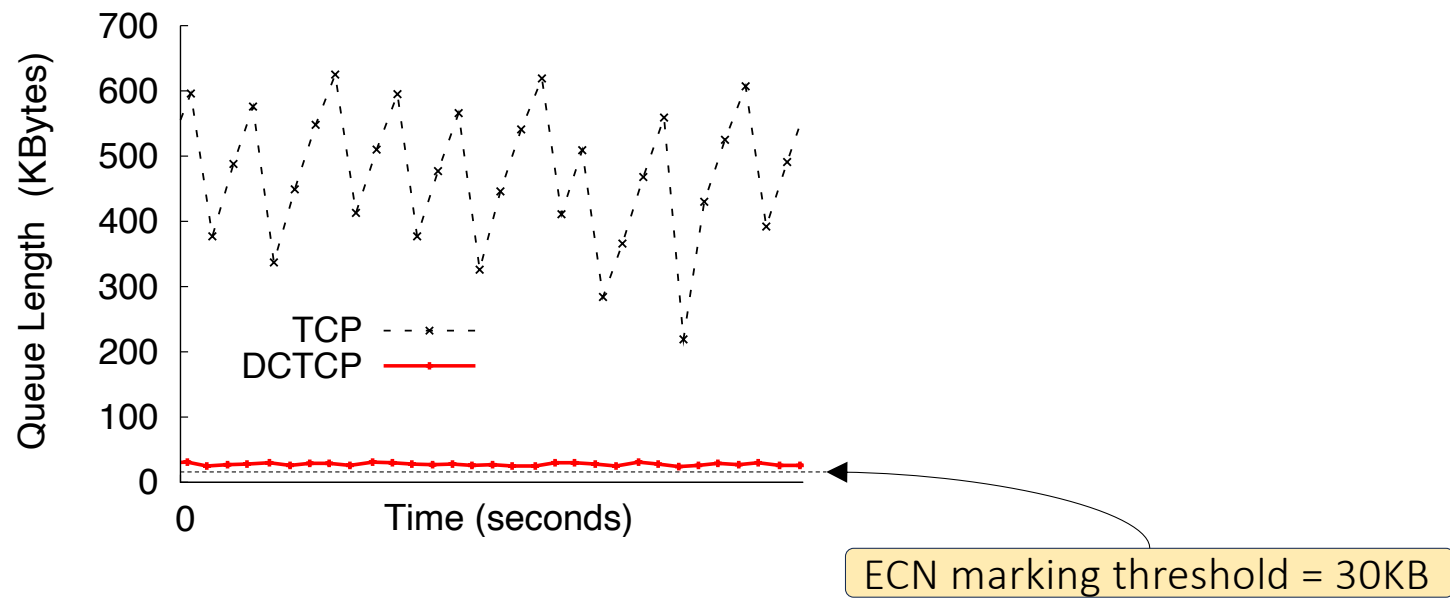
$$F = \frac{\# \text{ of marked ACKs}}{\text{Total \# of ACKs}}$$

g is the weight given to new samples

ECN Marks	TCP	DCTCP
1 0 1 1 1 1 0 1 1 1	Cut window by 50%	Cut window by 40%
0 0 0 0 0 0 0 0 0 1	Cut window by 50%	Cut window by 5%

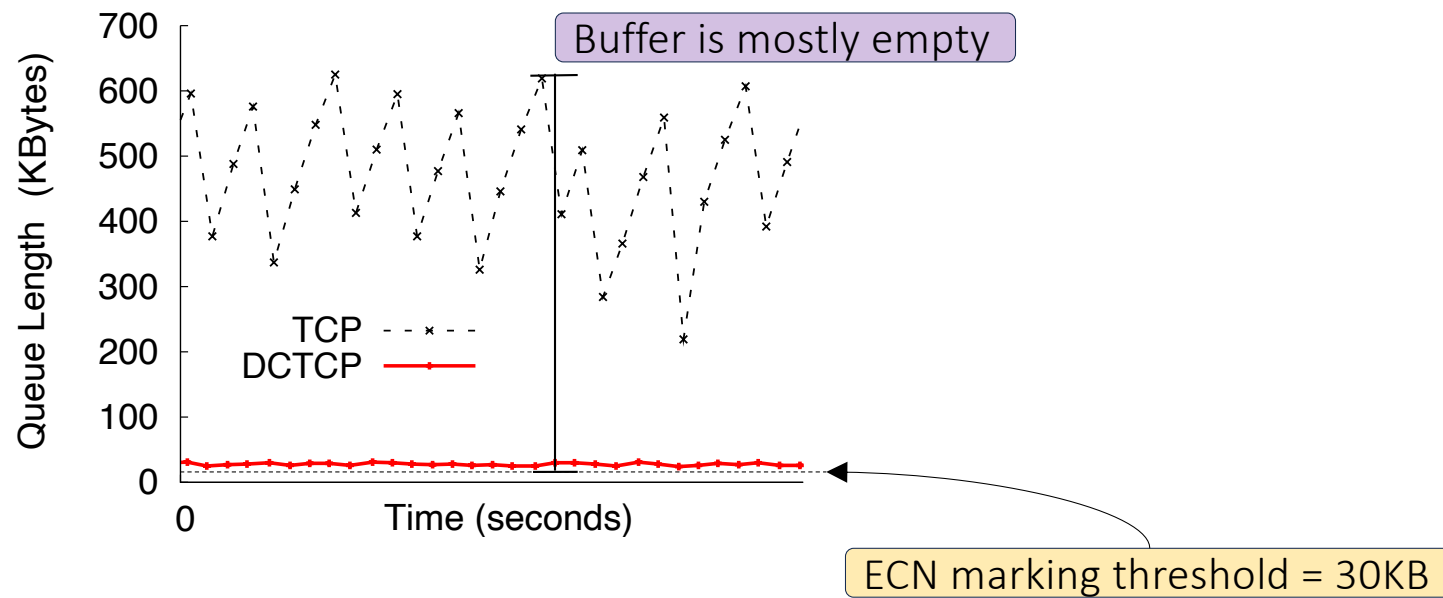
DCTCP vs TCP

2 flows, Broadcom 1Gbps Switch, 700KB buffer



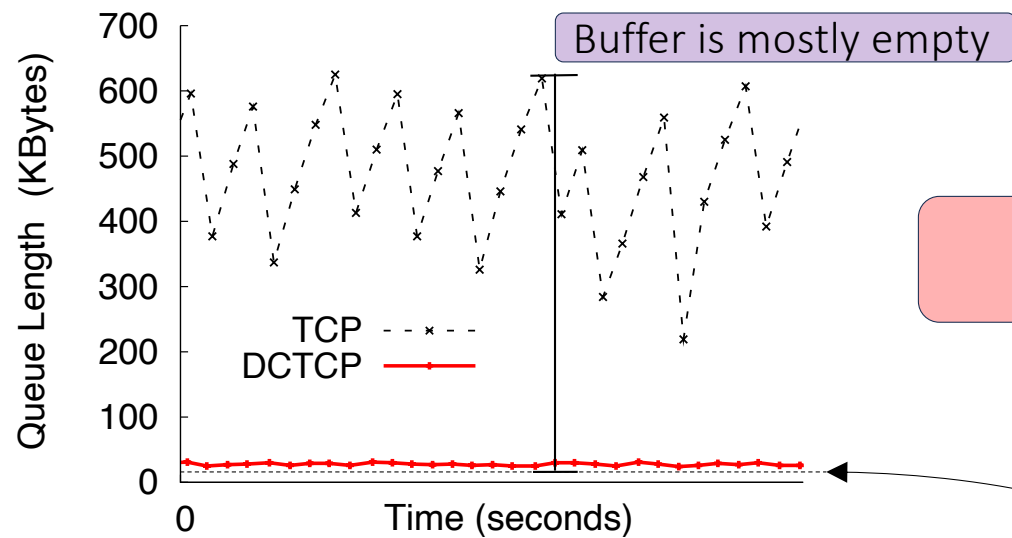
DCTCP vs TCP

2 flows, Broadcom 1Gbps Switch, 700KB buffer



DCTCP vs TCP

2 flows, Broadcom 1Gbps Switch, 700KB buffer



Buffer is mostly empty

DCTCP mitigates Incast by creating a large buffer headroom

ECN marking threshold = 30KB

Why it works?

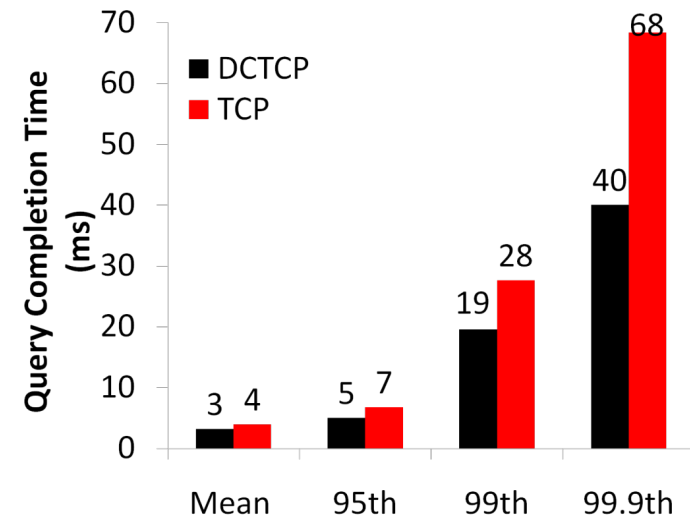
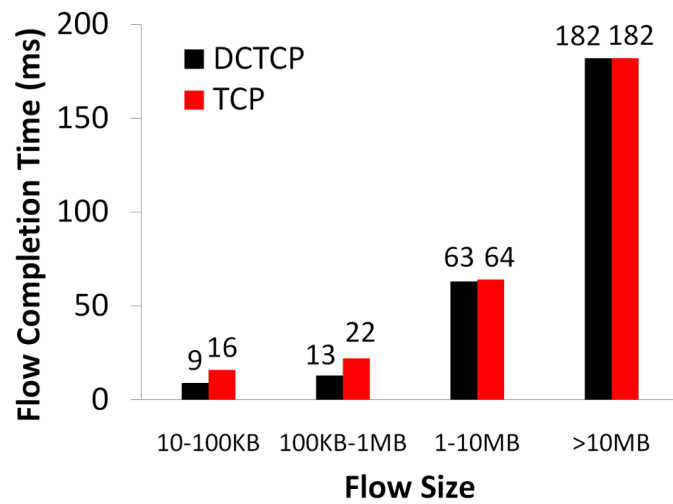
- Low Latency
 - Small buffer occupancies → low queuing delay
- High Throughput
 - ECN averaging → smooth rate adjustments, low variance
- High Burst Tolerance
 - Large buffer headroom → bursts fit
 - Aggressive marking → sources react before packets are dropped

Why it works?

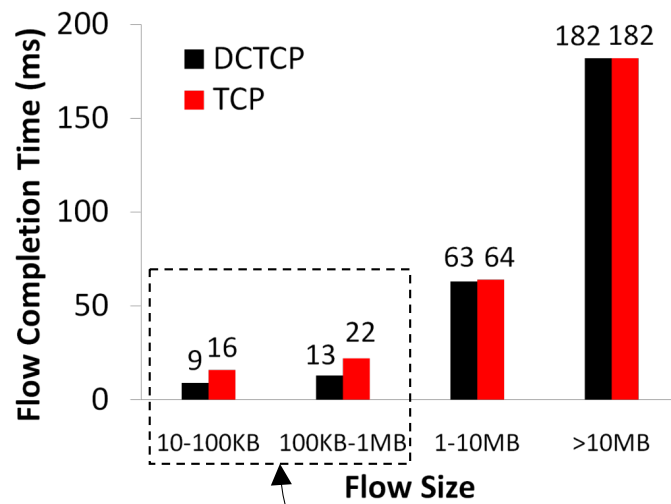
- Low Latency
 - Small buffer occupancies → low queuing delay
- High Throughput
 - ECN averaging → smooth rate adjustments, low variance
- High Burst Tolerance
 - Large buffer headroom → bursts fit
 - Aggressive marking → sources react before packets are dropped

In the DCTCP paper, you can find the mathematical analysis on how to choose the ECN threshold value and how to fine tune α

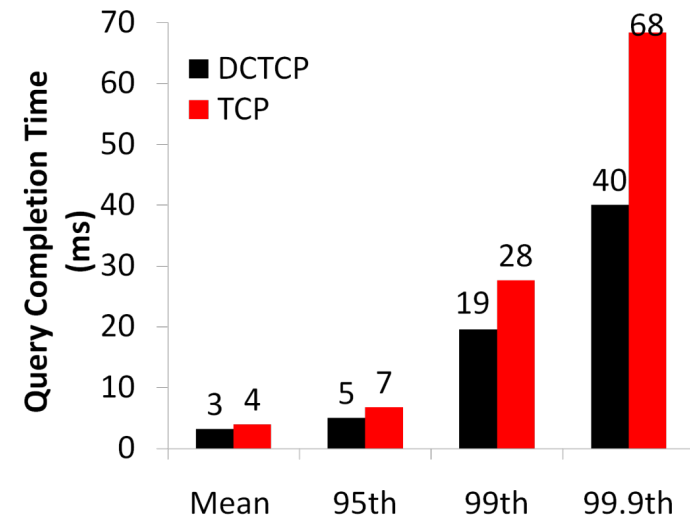
DCTCP in action



DCTCP in action

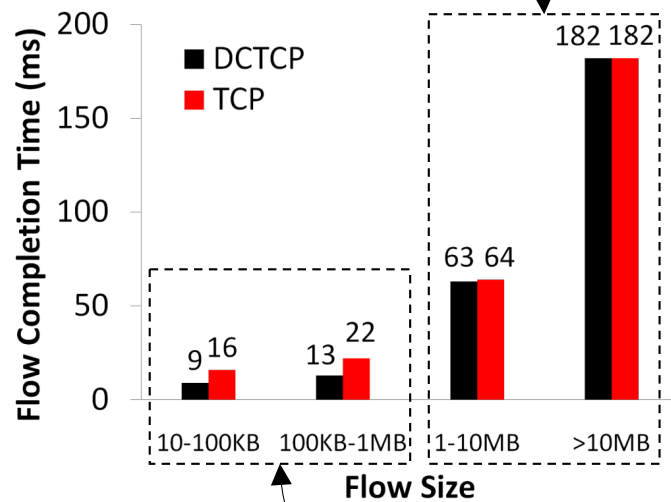


Low latency for short flows

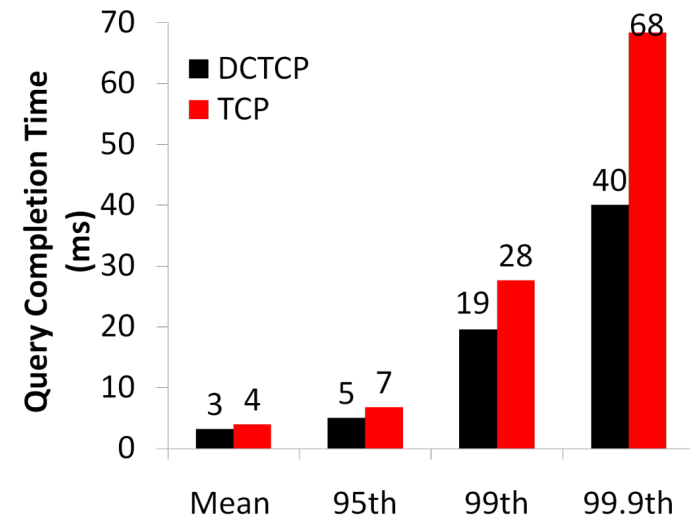


DCTCP in action

High throughput for long flows



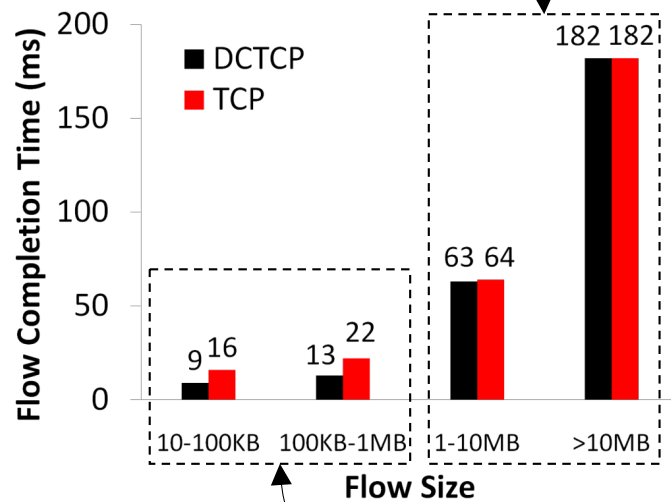
Low latency for short flows



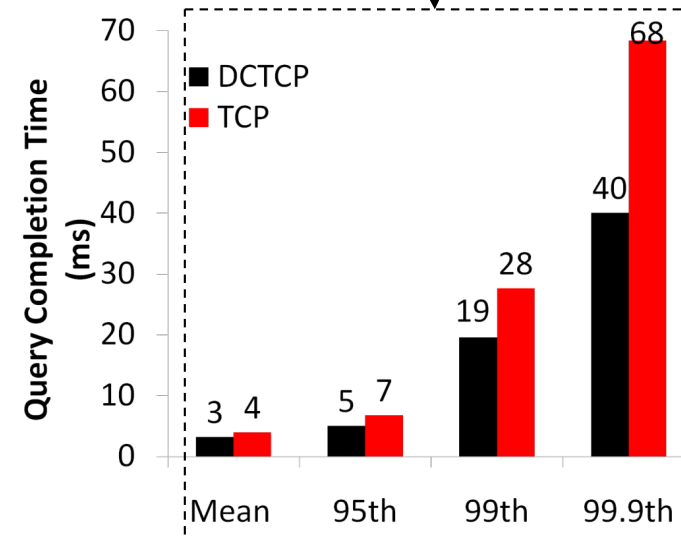
DCTCP in action

High burst tolerance for query flows
(good behavior in case of incast)

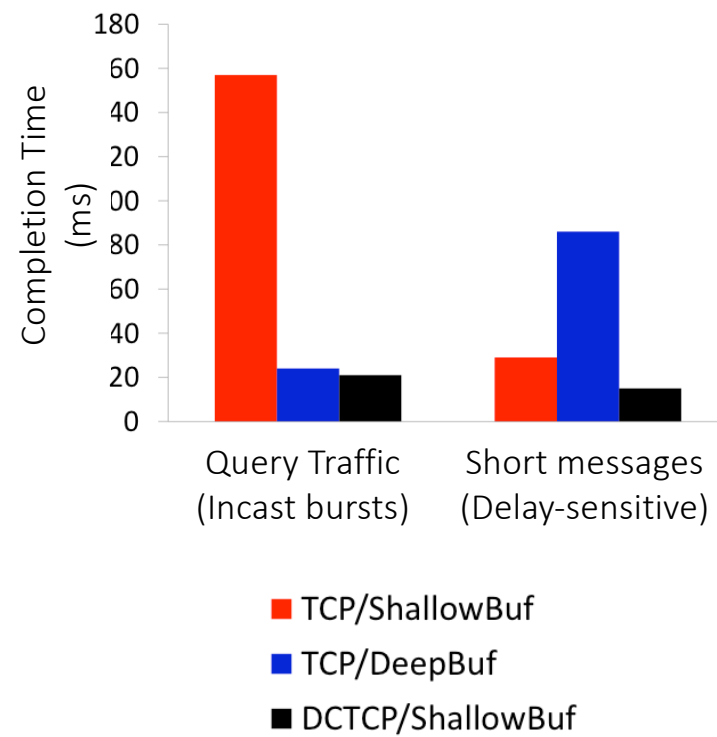
High throughput for long flows



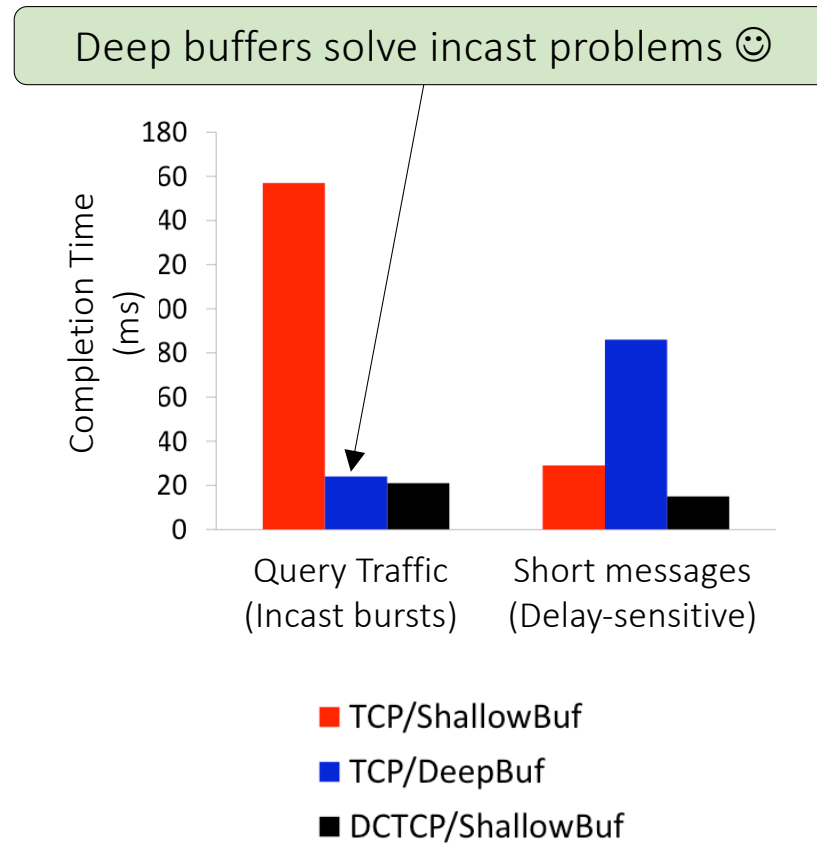
Low latency for short flows



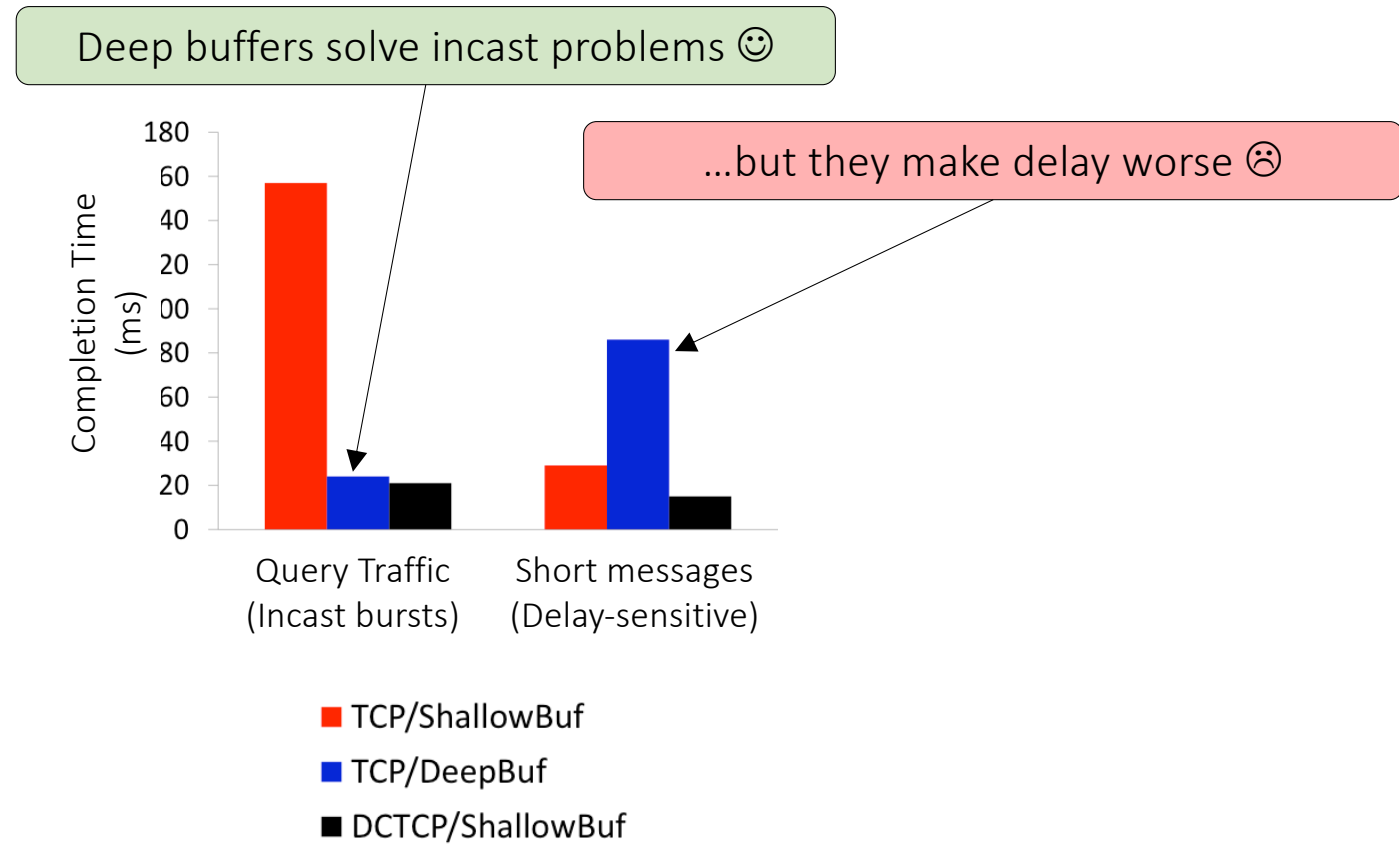
DCTCP in action



DCTCP in action



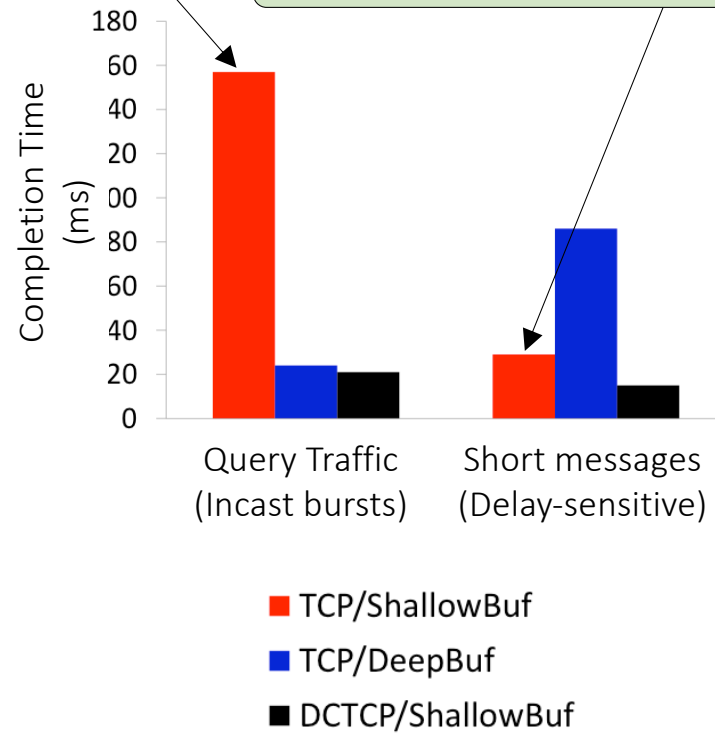
DCTCP in action



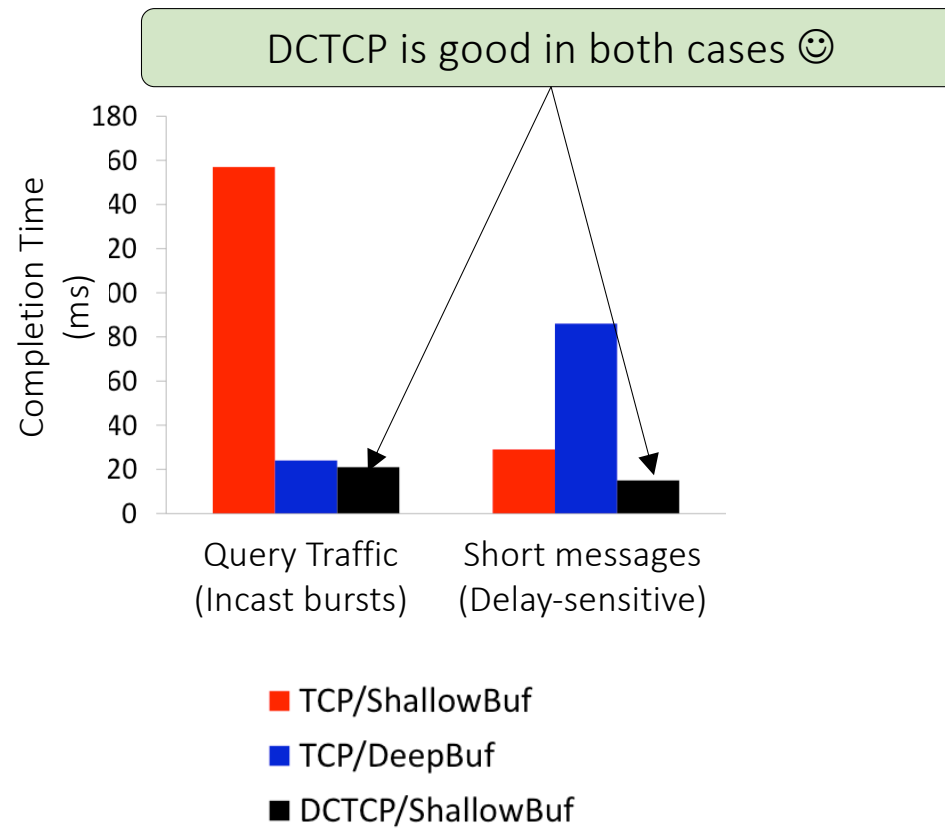
DCTCP in action

...but bad for incast workloads ☹️

Shallow buffers are good for latency 😊



DCTCP in action



Can we do better?

Can we do better?

- DCTCP improves latency but it is not optimal
 - latency-sensitive parallel short flows can get stuck waiting behind resource-intensive, long flows in switch queues

Idea: we shall handle flow prioritization explicitly at switches!

pFabric

pFabric: Minimal Near-Optimal Datacenter Transport

Mohammad Alizadeh^{†‡}, Shuang Yang[†], Milad Sharif[†], Sachin Katti[†],
Nick McKeown[†], Balaji Prabhakar[†], and Scott Shenker[§]

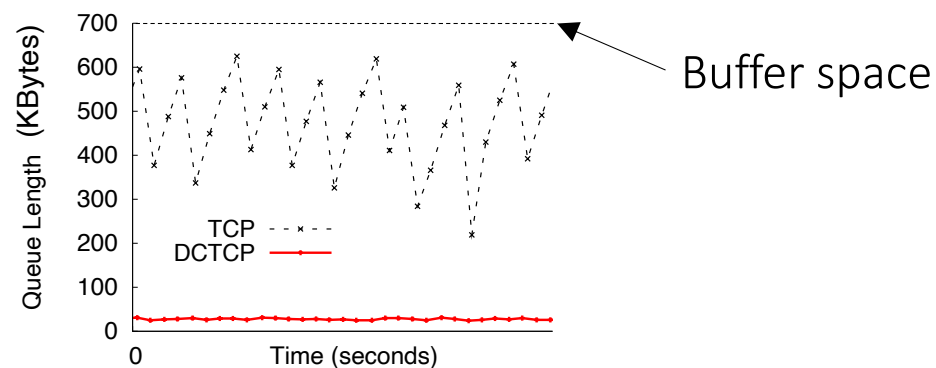
[†]Stanford University [‡]Insieme Networks [§]U.C. Berkeley / ICSI
{alizade, shyang, msharif, skatti, nickm, balaji}@stanford.edu shenker@icsi.berkeley.edu

- A solution that decouples rate control and flow scheduling enabling a new transport protocol that provides near-optimal flow completion times
- Need specific functions at switches
- Work presented at ACM SIGCOMM 2013 (Hong Kong)

Transport in datacenters

- Goal: complete flows quickly 😊
- Requires scheduling flows such that:
 - High throughput for large flows
 - Fabric latency (no queueing delays) for small flows

This is where DCTCP can fail!



pFabric idea in a single slide

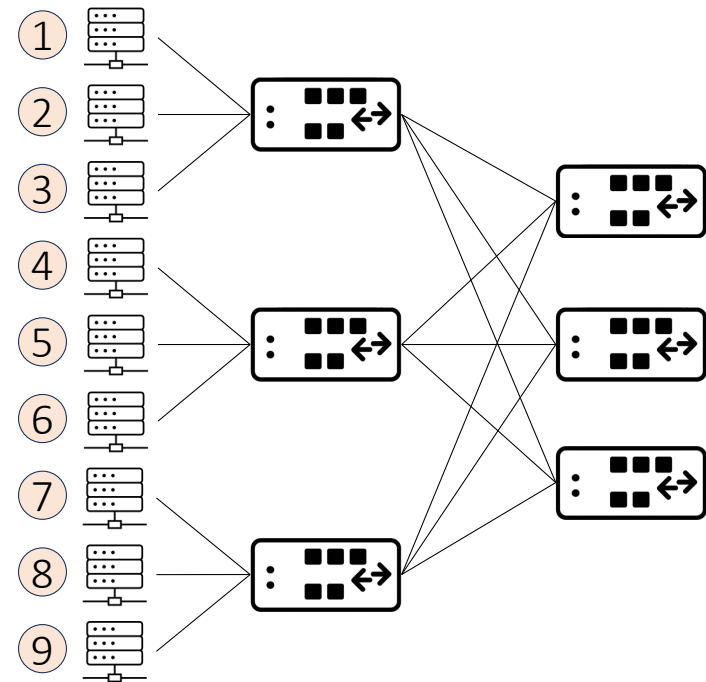
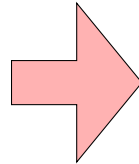
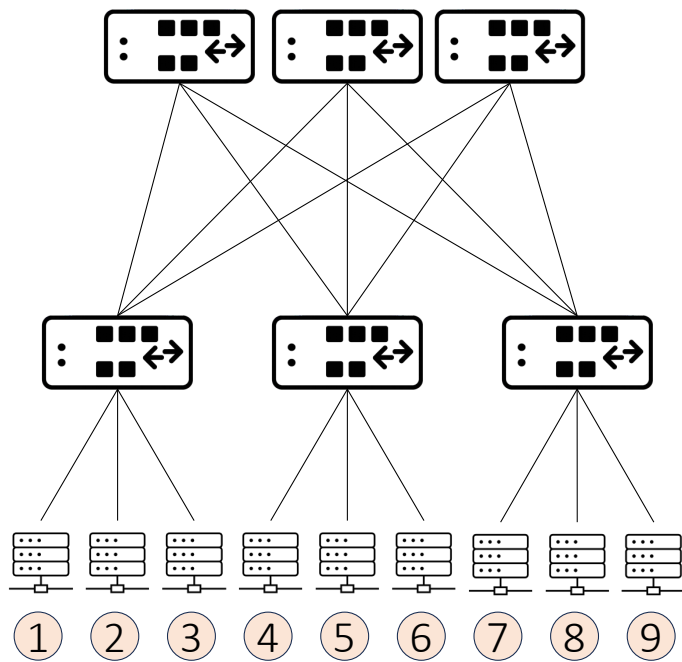
- Packets carry a single priority number
 - Example: remaining flow size
- pFabric switches
 - Very small buffers (we do not need headroom as in DCTCP, just small buffers)
 - Process highest priority and drop lowest priority packets
- pFabric hosts
 - Send/retransmit aggressively
 - Minimal rate control: just try to prevent congestion collapse

pFabric conceptual model

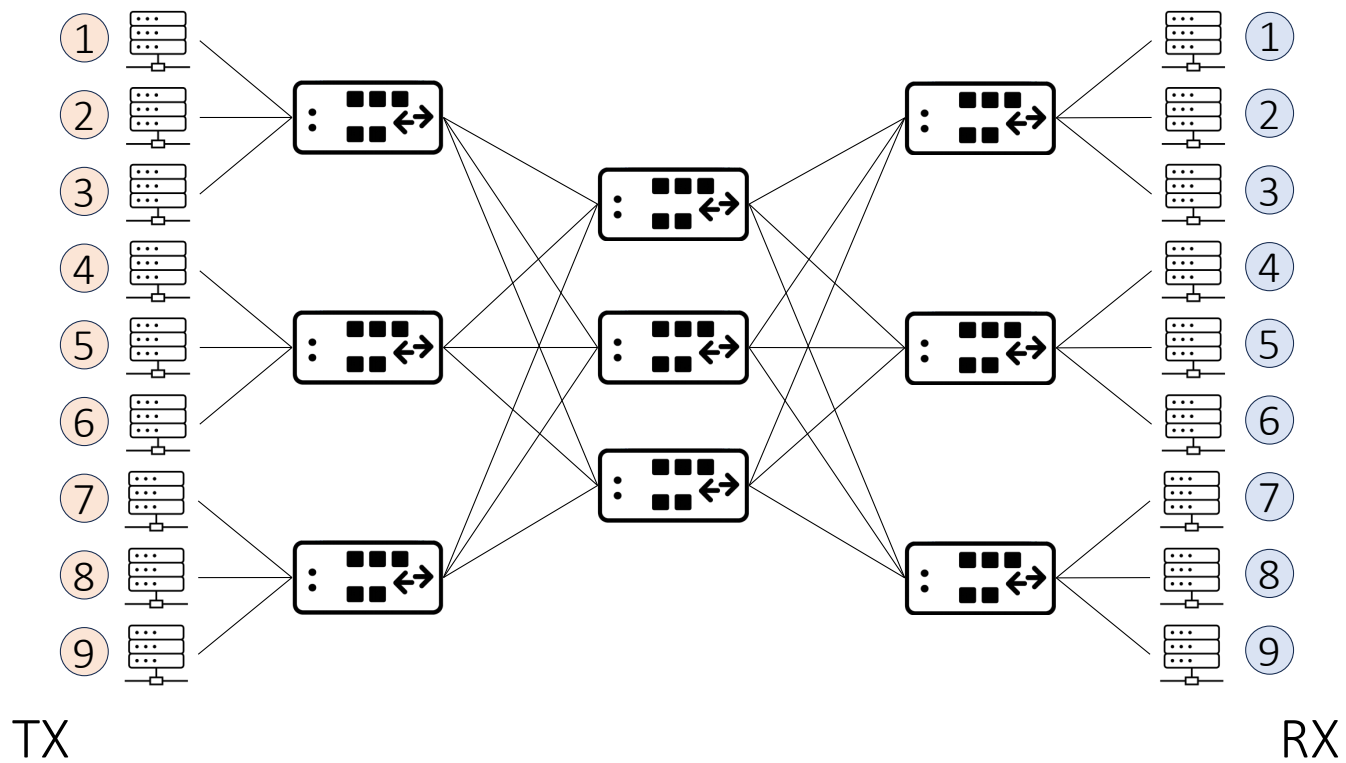
- The starting point:
 - We need to orchestrate network resource allocation to meet *overall computing objectives*
 - Network fabric shall be designed to schedule flows to maximize application-level objectives (flow completion time)
 - Traditional metrics such as throughput or fairness (typical of TCP) are less important

To do that let's abstract the entire fabric as a giant switch

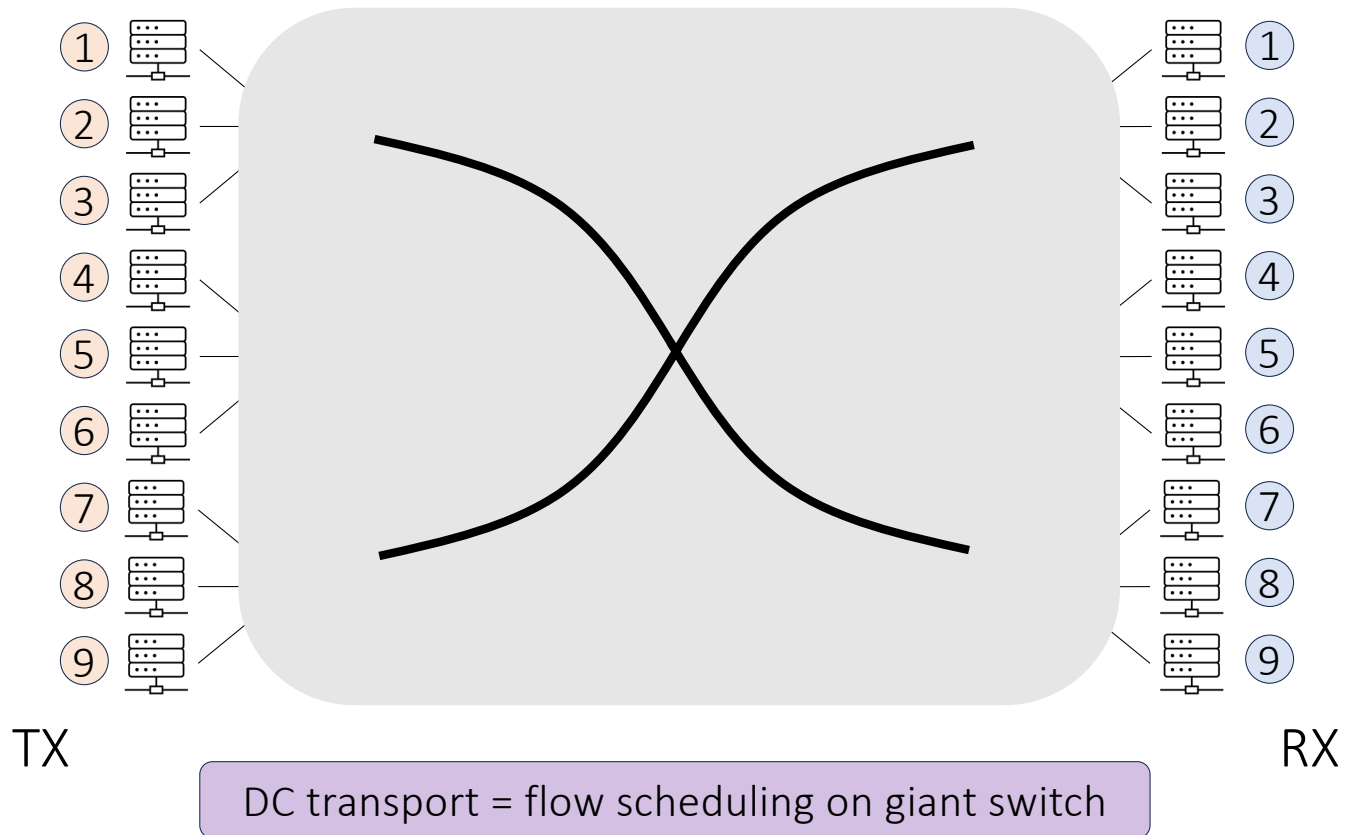
pFabric conceptual model



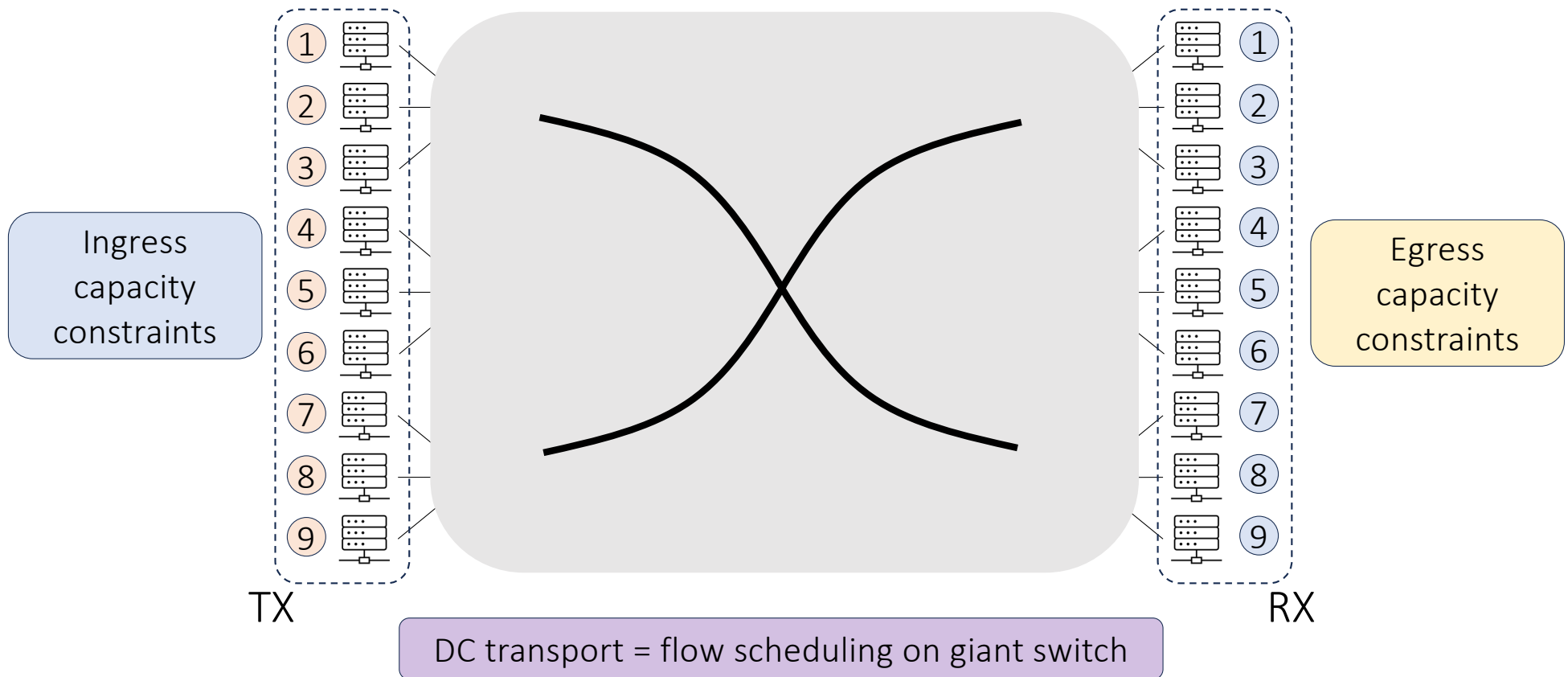
pFabric conceptual model



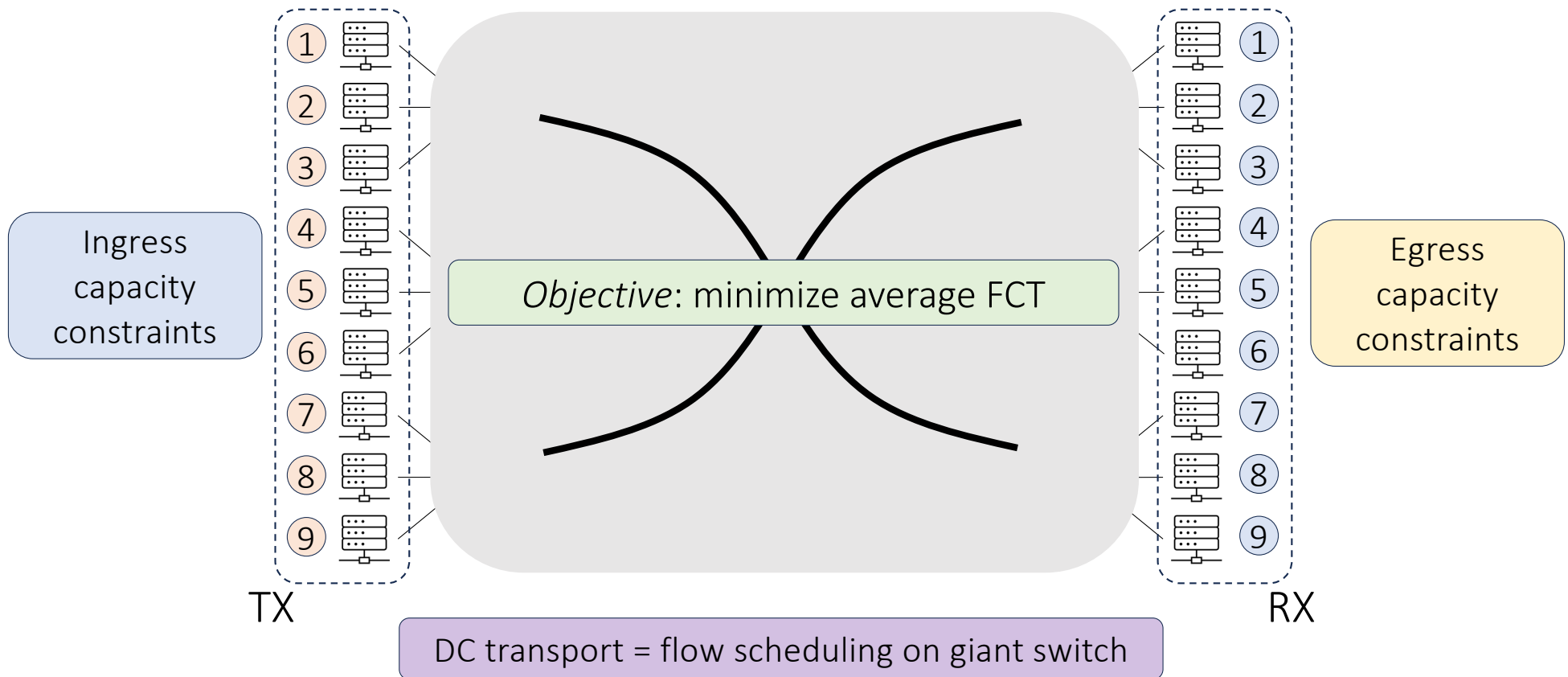
pFabric conceptual model



pFabric conceptual model



pFabric conceptual model



Optimal flow scheduling

**A PROOF OF THE OPTIMALITY OF THE SHORTEST
REMAINING PROCESSING TIME DISCIPLINE**

Linus Schrage

University of Chicago, Chicago, Illinois

(Received January 17, 1968)

- The optimal algorithm for minimizing average FCT when scheduling over a single link is the Shortest Remaining Processing Time (SRPT) which schedules the flow that has the least work remaining

Optimal flow scheduling

A PROOF OF THE OPTIMALITY OF THE SHORTEST
REMAINING PROCESSING TIME DISCIPLINE

Linus Schrage

University of Chicago, Chicago, Illinois

(Received January 17, 1968)

- The optimal algorithm for minimizing average FCT when scheduling over a single link is the Shortest Remaining Processing Time (SRPT) which schedules the flow that has the least work remaining

Problem: here we are not scheduling over a single link, rather an entire fabric

Optimal flow scheduling

A PROOF OF THE OPTIMALITY OF THE SHORTEST
REMAINING PROCESSING TIME DISCIPLINE

Linus Schrage

University of Chicago, Chicago, Illinois

(Received January 17, 1968)

- The optimal algorithm for minimizing average FCT when scheduling over a single link is the Shortest Remaining Processing Time (SRPT) which schedules the flow that has the least work remaining

Problem: here we are not scheduling over a single link, rather an entire fabric

Insight: prioritizing small flows over large flows end-to-end across the fabric can provide near-ideal average FCT!

Finishing Flows Quickly with Preemptive Scheduling

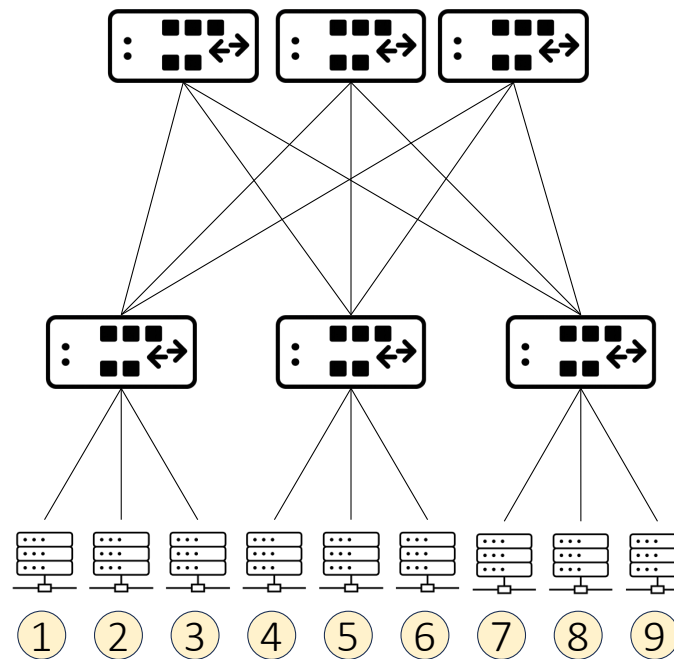
Chi-Yao Hong
UIUC
cyhong@illinois.edu

Matthew Caesar
UIUC
caesar@illinois.edu

P. Brighten Godfrey
UIUC
pbg@illinois.edu

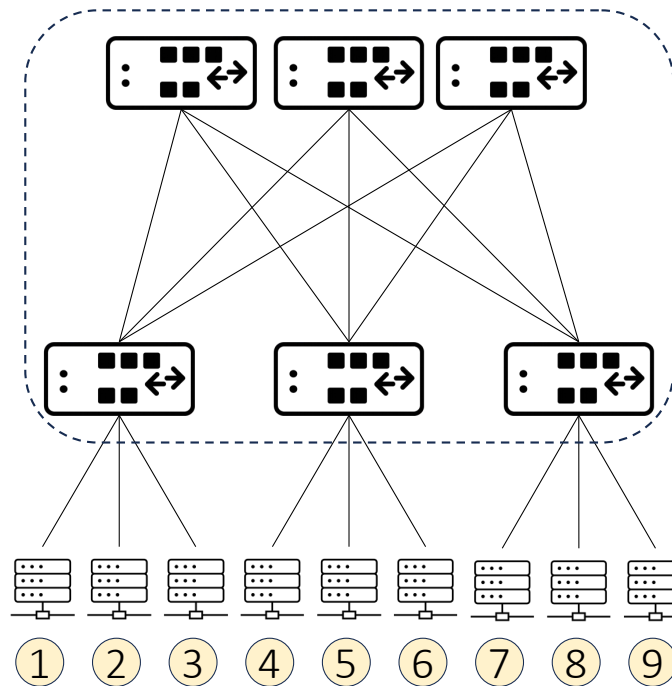
pFabric in a slide

Key idea: decouple flow scheduling from rate control



pFabric in a slide

Key idea: decouple flow scheduling from rate control



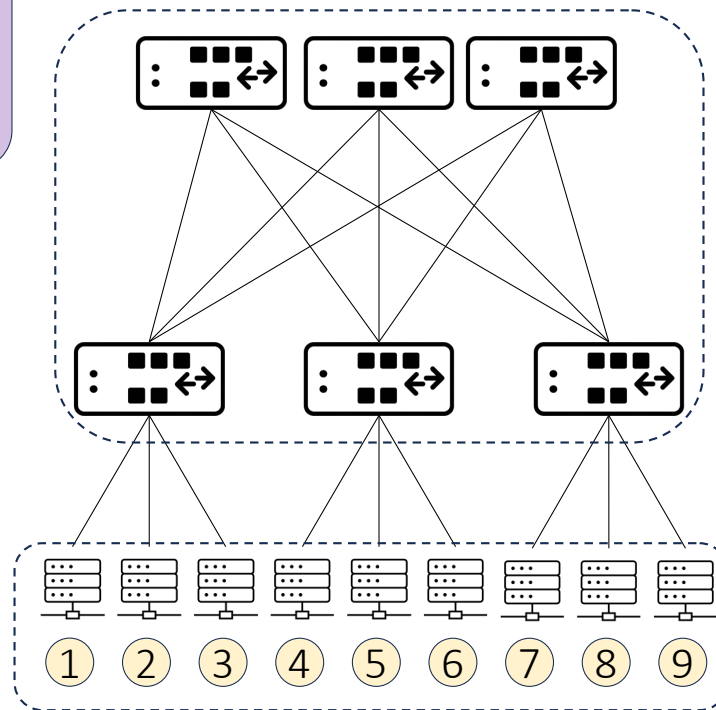
switches implement flow scheduling via local mechanisms

pFabric in a slide

Key idea: decouple flow scheduling from rate control

switches implement flow scheduling via local mechanisms

Hosts implement simple rate control to avoid high packet loss

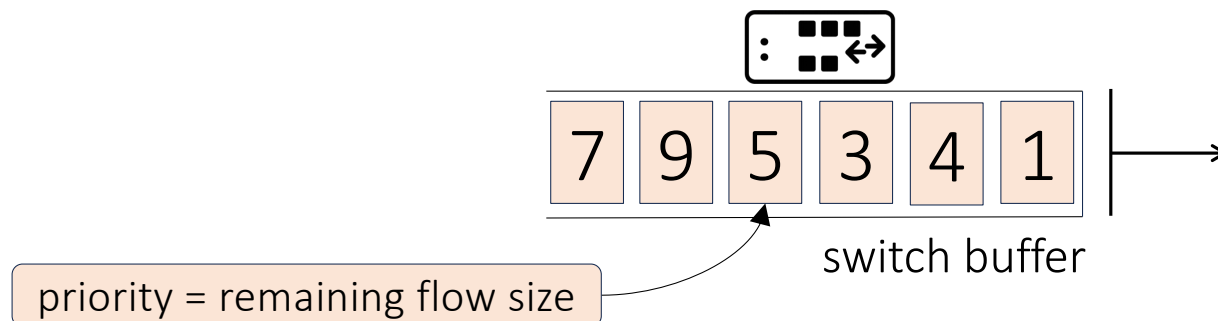


pFabric switch

- Priority scheduling
 - Send highest priority packet first
- Priority dropping
 - Drop lowest priority packets first

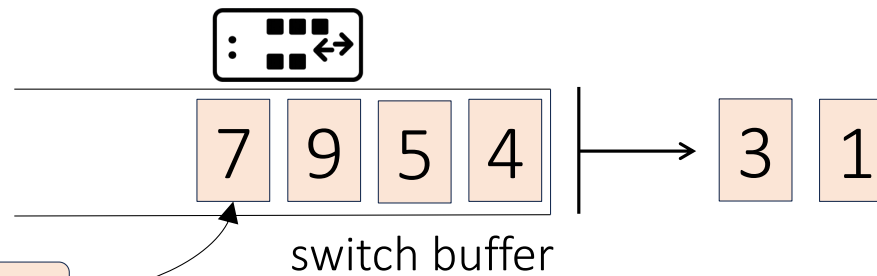
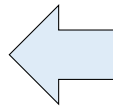
pFabric switch

- Priority scheduling
 - Send highest priority packet first
- Priority dropping
 - Drop lowest priority packets first



pFabric switch

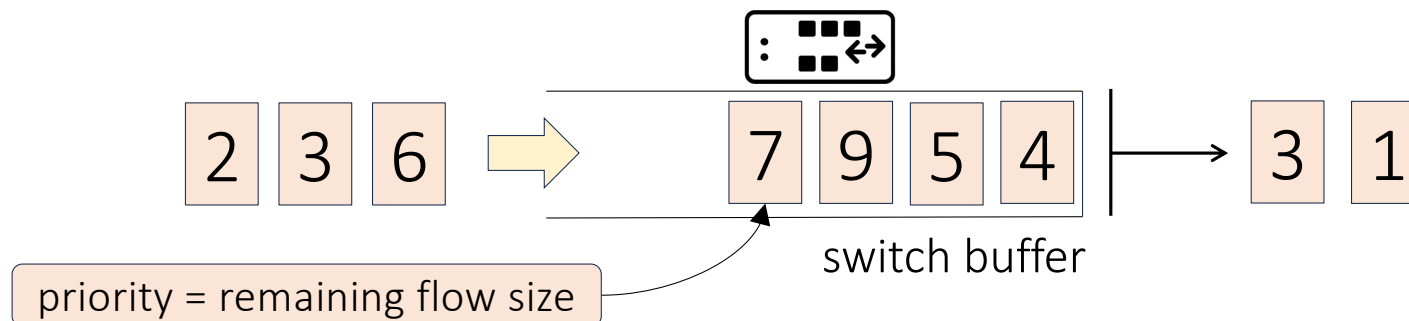
- Priority scheduling
 - Send highest priority packet first
- Priority dropping
 - Drop lowest priority packets first



priority = remaining flow size

pFabric switch

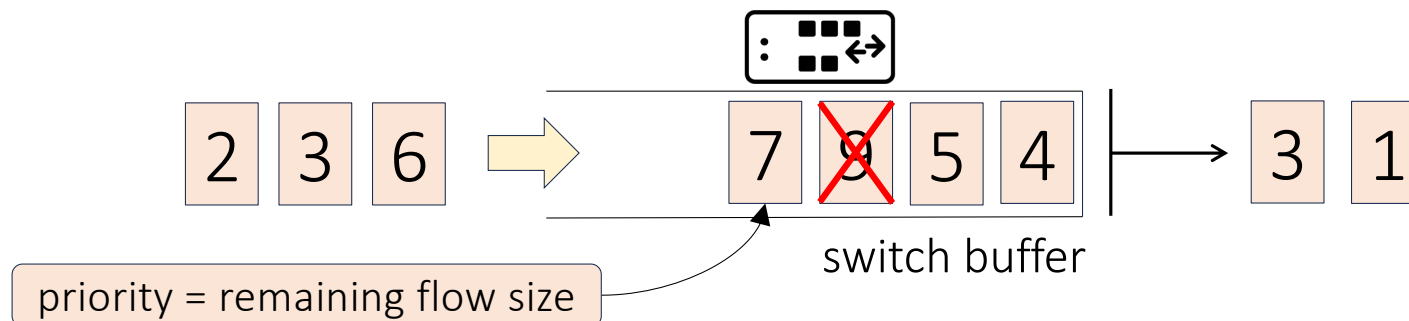
- Priority scheduling
 - Send highest priority packet first
- Priority dropping
 - Drop lowest priority packets first



pFabric switch

- Priority scheduling
 - Send highest priority packet first

- Priority dropping
 - Drop lowest priority packets first



pFabric example

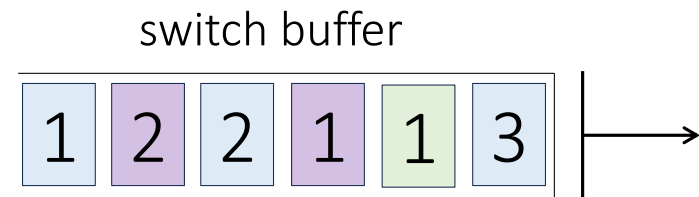
Host A needs to send 3KB to Host B (blue packets)

Let's consider an MTU of 1KB and do not take into account headers

packet 1 will have priority set to 3

packet 2 will have priority set to 2

packet 3 will have priority set to 1



pFabric example

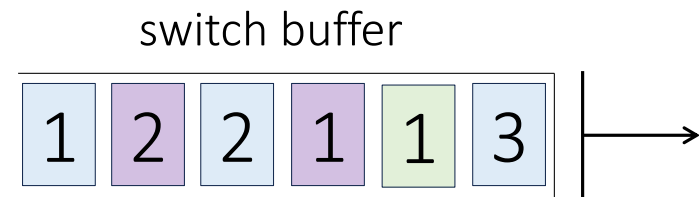
Host A needs to send 3KB to Host B (blue packets)

Let's consider an MTU of 1KB and do not take into account headers

packet 1 will have priority set to 3

packet 2 will have priority set to 2

packet 3 will have priority set to 1



What's the problem with blue flow?

pFabric example

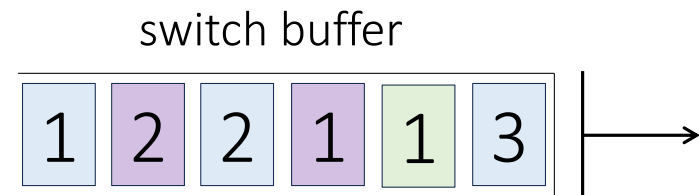
Host A needs to send 3KB to Host B (blue packets)

Let's consider an MTU of 1KB and do not take into account headers

packet 1 will have priority set to 3

packet 2 will have priority set to 2

packet 3 will have priority set to 1



What's the problem with blue flow?

Starvation! How to solve this?

pFabric example

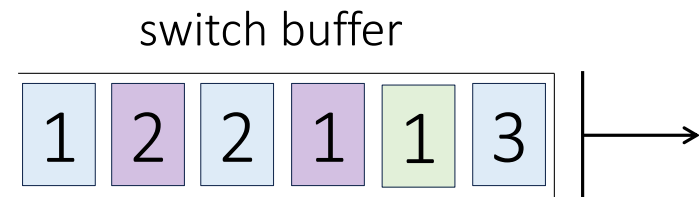
Host A needs to send 3KB to Host B (blue packets)

Let's consider an MTU of 1KB and do not take into account headers

packet 1 will have priority set to 3

packet 2 will have priority set to 2

packet 3 will have priority set to 1



What's the problem with blue flow?

Starvation! How to solve this?

pFabric dequeues the earliest packet from the flow that has the highest priority packet in the queue

pFabric switch complexity

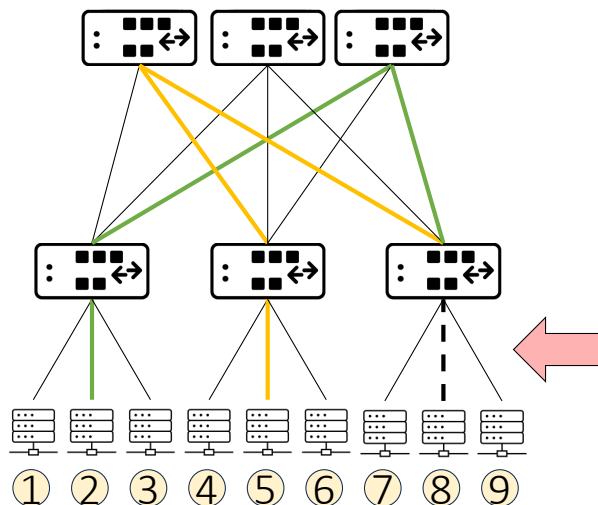
- Use very small buffers
 - 2xBDP per-port (with a capacity=10Gbps and RTT=15 μ s, buffer is approximately 30KB)
- Priority scheduling
 - We need just a binary comparator tree
 - Easy to implement in hardware, not much additional cost

pFabric rate control

- We do not need to worry about queues
 - Priority scheduling helps and we have controlled latency for small flows by design
 - This greatly simplify rate control 😊

pFabric rate control

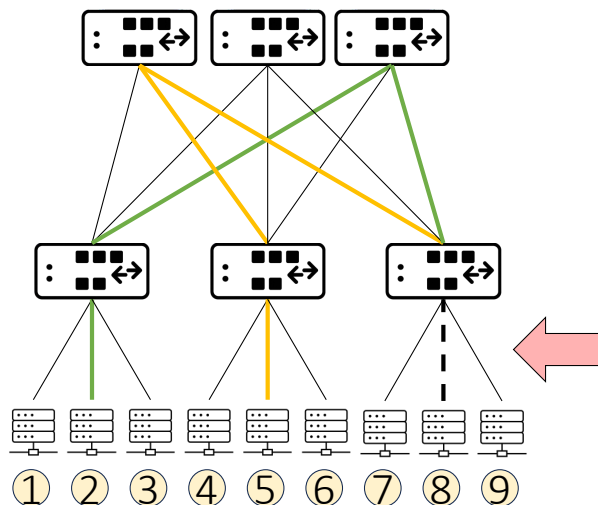
- We need to worry about wasted resources
 - A packet traverses multiple hops only to be dropped at the last one
 - This wastes bandwidth that could have been used to transmit other packets



if two elephant flows sending at line rate collide at a last hop, half the bandwidth they consume on the upstream links is wasted

pFabric rate control

- We need to worry about wasted resources
 - A packet traverses multiple hops only to be dropped at the last one
 - This wastes bandwidth that could have been used to transmit other packets



We need to prevent congestion collapse when elephants collide

if two elephant flows sending at line rate collide at a last hop, half the bandwidth they consume on the upstream links is wasted

pFabric rate control

- Minimal version of the TCP algorithm
 - Start at line rate (use an initial window size equal to the BDP of the link)
 - There are no fast retransmits or any other similar mechanisms. Packet drops are only detected by timeouts (fixed and small, e.g., $3 \times$ the fabric RTT)
 - Upon a timeout, the flow enters into slow start and *ssthresh* is set to half the window size before the timeout occurred

Why pFabric works?

- When a packet is dropped, it has the lowest priority among all buffered packets
 - Even if it were not dropped, it would only be serviced after all previously buffered packets have left the switch

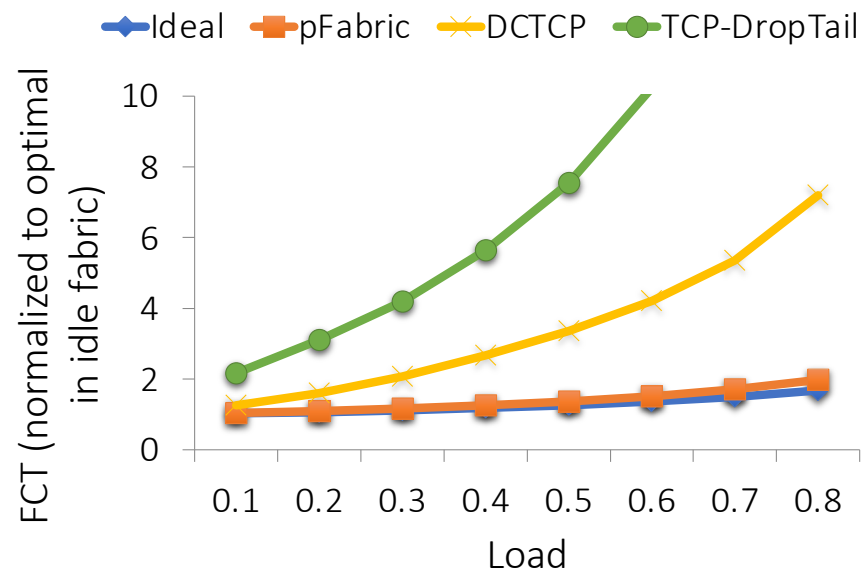
Why pFabric works?

- When a packet is dropped, it has the lowest priority among all buffered packets
 - Even if it were not dropped, it would only be serviced after all previously buffered packets have left the switch

...as a consequence

- A packet can safely be dropped if rate control is aggressive and ensures that it retransmits the packet before all the existing packets depart the switch
 - This can easily be achieved if the buffer size is at least one BDP and hence takes more than a RTT to drain, providing the end-host enough time to detect and retransmit dropped packets

Overall average FCT



Ideal is ideal 😊

- Centralized with full view of flows
- No rate-control dynamics
- No buffering
- No pkt drops
- No load-balancing inefficiency

now?

Is congestion control in datacenter solved then?

now?

Annulus: A Dual Congestion Control Loop for Datacenter and WAN Traffic Aggregates

Ahmed Saeed[◊], Varun Gupta[†], Prateesh Goyal[◊], Milad Sharif[‡], Rong Pan[§], Mosta
Ellen Zegura[◊], Keon Jang[★], Mohammad Alizadeh[◊], Abdul Kabbani[★], Amin V
[◊]MIT CSAIL, [†]AT&T, [‡]SambaNova, [§]Intel, [◊]Georgia Tech, [★]MPI-SWS, [★]Google

Swift: Delay is Simple and Effective for Congestion Control in the Datacenter

Gautam Kumar, Nandita Dukkkipati, Keon Jang (MPI-SWS)^{*}, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri,
Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat
Google LLC

Is congestion control in datacenter solved then?

TIMELY: RTT-based Congestion Control for the Datacenter

Radhika Mittal^{*}(UC Berkeley), Vinh The Lam, Nandita Dukkkipati, Emily Blem, Hassan Wass
Monia Ghobadi^{*}(Microsoft), Amin Vahdat, Yaogong Wang, David Wetherall, David Zats

Google, Inc.



PBE-CC: Congestion Control via Endpoint-Centric, Physical-Layer Bandwidth Measurements

Yaxiong Xie, Fan Yi, Kyle Jamieson
Department of Computer Science, Princeton University
{yaxiong, fanyi, kyle}@cs.princeton.edu

ACC: Automatic ECN Tuning for High-Speed Datacenter Networks

Siyu Yan[†], Xiaoliang Wang[★], Xiaolong Zheng[†], Yinben Xia[†]
Derui Liu[★], Weishan Deng[†]
[†] Huawei Technologies Co., Ltd [★] Nanjing University

Re-architecting datacenter networks and stacks for low latency and high performance

Mark Handley
University College London
London, UK
m.handley@cs.ucl.ac.uk

Costin Raiciu
Alexandru Agache
Andrei Voinescu
University Politehnica of Bucharest
Bucharest, Romania
firstname.lastname@cs.pub.ro

Andrew W. Moore
Gianni Antichi
Marcin Wójcik
University of Cambridge
Cambridge, UK
firstname.lastname@cl.cam.ac.uk

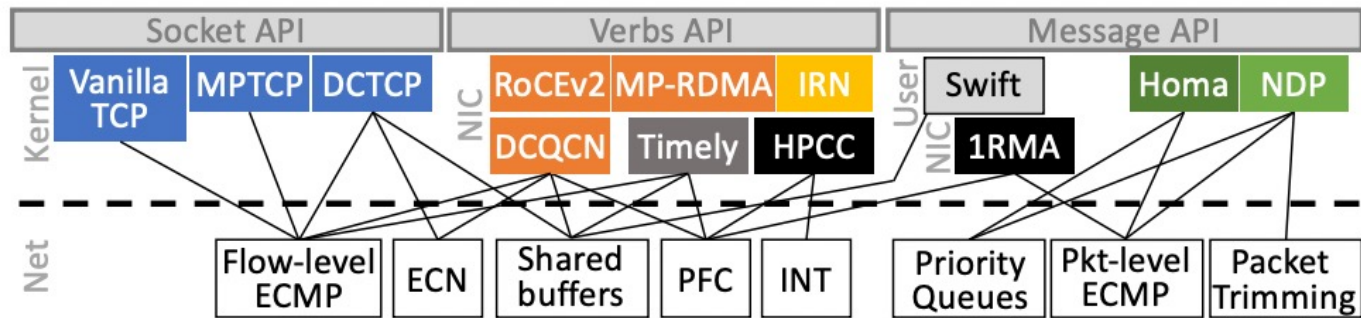
HPCC: High Precision Congestion Control

Yuliang Li[◊], Rui Miao[★], Hongqiang Harry Liu[★], Yan Zhuang[★], Fei Feng[★], Lingbo Tang[★], Zheng Cao[★], Ming Zhang[★],
Frank Kelly[◊], Mohammad Alizadeh[★], Minlan Yu[◊]
[★] Alibaba Group[★], [◊] Harvard University[◊], [◊] University of Cambridge[◊], [★] Massachusetts Institute of Technology[★]

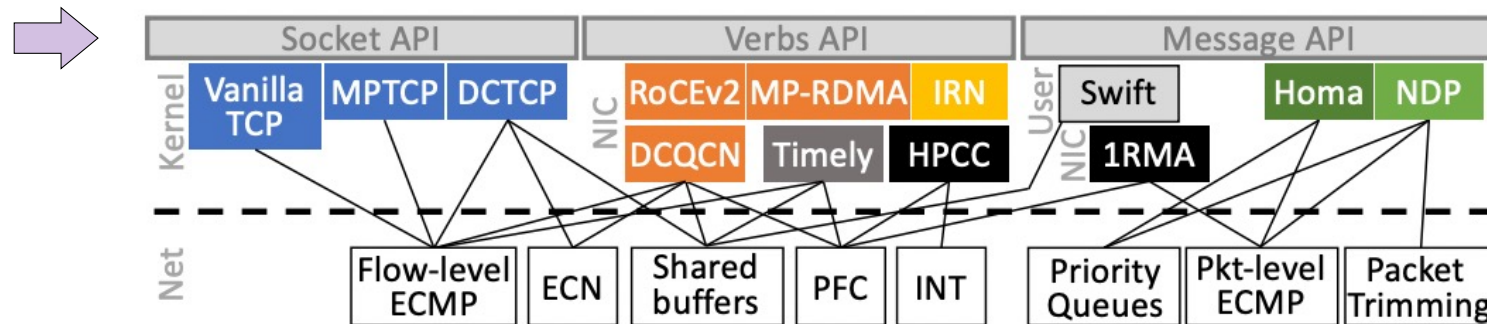
Homa: A Receiver-Driven Low-Latency Transport Protocol Using Network Priorities

Behnam Montazeri, Yilong Li, Mohammad Alizadeh[†], and John Ousterhout
Stanford University, [†]MIT

Fragmentation of datacenter networking

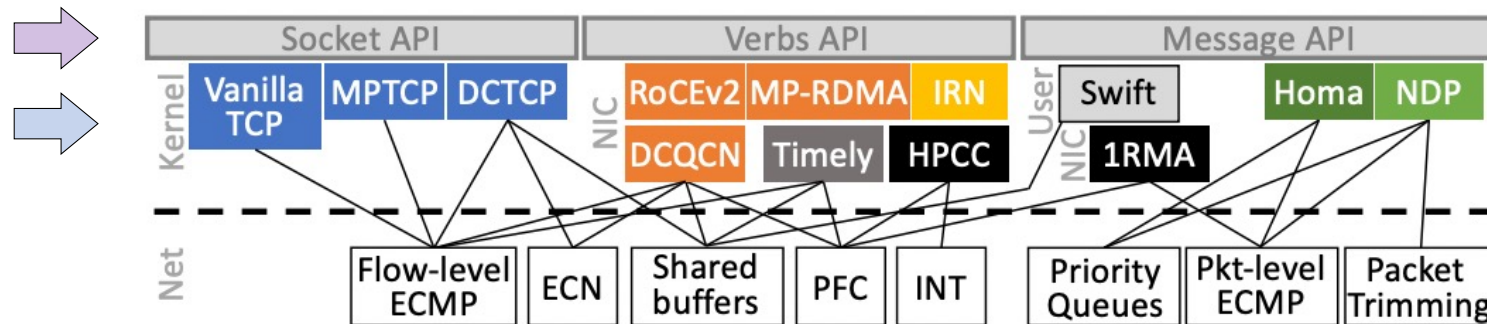


Fragmentation of datacenter networking



They might be characterized by different APIs exposed to applications

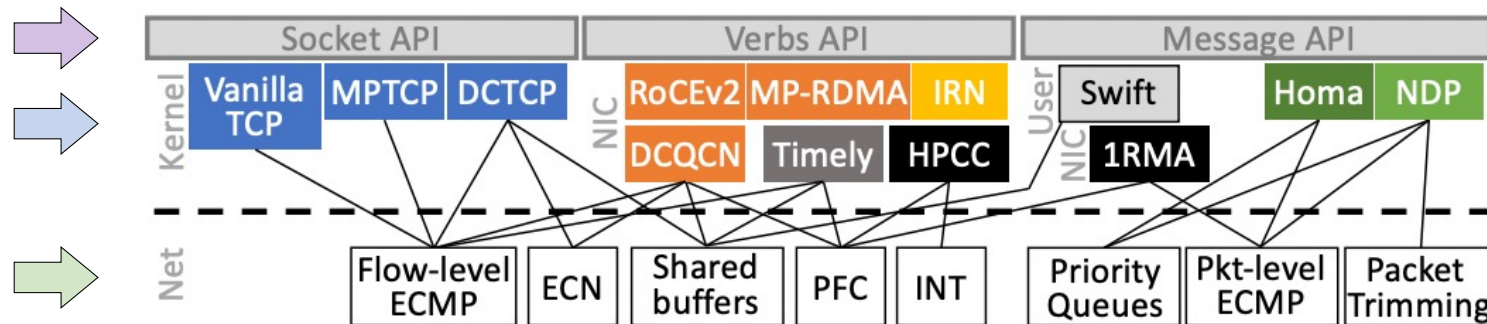
Fragmentation of datacenter networking



They might be characterized by different APIs exposed to applications

Some are implemented in kernel, others at user-space or in the NIC

Fragmentation of datacenter networking



They might be characterized by different APIs exposed to applications

Some are implemented in kernel, others at user-space or in the NIC

Different techniques: multi-path, ECN, priorities, packet trimming, INT